

Opinosis 텍스트 데이터 기반 토픽 군집화 최종 분석 보고서

분석팀: 이해현, 이진희, 김소현, 전유경, 권용우

보고서 작성: 이해현

목차

- [0. 초록](#)
- [1. 서론](#)
 - 1.1 연구 배경 및 필요성
 - 1.2 과제 목적
- [2. 데이터 소개](#)
 - 2.1 데이터 출처 및 구성
 - 2.2 데이터 규모 및 기본 통계
 - 2.3 도메인별 분포
- [3. 탐색적 데이터 분석 \(EDA\)](#)
 - 3.1 기초 텍스트 통계
 - 3.2 상위 빈출 단어 분석
- [4. 데이터 전처리 및 벡터화](#)
 - 4.1 텍스트 전처리 파이프라인
 - 4.2 텍스트 벡터화 기법 비교
- [5. 분석방법 \(Analysis Method\)](#)
 - 5.1 전처리·벡터화 필요성 및 과정 개요
 - 5.2 사용 모델 설명
 - 5.3 하이퍼파라미터 최적화 기법 설명
 - 5.4 실행환경
 - 5.5 코드 재사용 및 함수화
- [6. 모델링](#)
 - 6.1 차원 축소
 - 6.2 군집화 모델 비교
 - 6.3 하이퍼파라미터 최적화 (그리드 탐색)
 - 6.4 $k=5$ 세분화의 독립적 재현 검증 (전유경 파이프라인)
- [7. 결과](#)
 - 7.1 최종 파이프라인
 - 7.2 주요 발견사항
 - 7.3 비즈니스 활용 제언 (투-트랙 전략)
- [8. 종합 평가](#)
- [9. 결론](#)
 - 9.1 기술적 결론

- 9.2 실무적 시사점
- [10. 회고](#)
 - 10.1 종합 회고
 - 10.2 팀원별 느낀점 및 피드백
- [11. 참고문헌](#)
- [12. 부록: 함수코드](#)
 - 12.1 전처리 함수
 - 12.2 클러스터 해석 함수
 - 12.3 검증 함수

0. 초록

본 보고서는 UCI Machine Learning Repository가 공개한 *Opinosis* 리뷰 데이터셋(51개 문서, 전자기기·자동차·호텔 3개 도메인)을 대상으로, 비정형 텍스트를 전처리·벡터화한 뒤 비지도 군집화(KMeans, GMM, DBSCAN, HDBSCAN, Agglomerative)로 문서를 토픽별로 자동 분류하는 프로젝트를 정리한 것이다. 팀원 5명(이혜현·이진화·김소현·전유경·권용우)이 각자 독립적인 전처리·벡터화·모델링 파이프라인을 구축해 비교했으며, 동일한 하이퍼파라미터(`ngram_range=(1,2)`, `min_df=0.05`, `max_df=0.85`)를 설정해도 토큰나이저 구현(축약형 처리, 표제어 추출 여부 등) 차이로 최종 피쳐 수가 파이프라인마다 달랐다는 점이 본 프로젝트의 특징적 발견이었다. 그 결과 김소현의 파이프라인(7단계 Lemmatization + TF-IDF, 51×4,498 희소 행렬)에서 KMeans(k=3)가 **ARI/NMI = 1.0**으로 51개 문서를 도메인과 완전히 일치하게 분류했으며, 이혜현의 54개 파라미터 그리드서치와 전유경의 독립 k=5 파이프라인이 공통적으로 “3대 도메인이 다시 5개 세부 니즈로 갈라진다”는 세분화 구조를 재현했다. 반대로 DBSCAN·HDBSCAN은 51개라는 소규모 데이터셋 특성상 거리 수렴 현상과 노이즈 과다로 부적합함이 세 명의 독립 실험에서 공통 확인되었다. 결론적으로 벡터화 기법(Count vs TF-IDF) 선택이 모델 선택보다 성능에 더 큰 영향을 미쳤으며, 도메인 완전분류(k=3)와 세부 니즈 세분화(k=5)를 병행하는 투-트랙 전략을 최종 제언으로 제시한다.

1. 서론

1.1 연구 배경 및 필요성

Opinosis는 UCI Machine Learning Repository에서 제공하는 공개 텍스트 데이터셋으로, 아마존 킨들·가민 GPS·넷북 등 전자기기, 혼다·도요타 자동차, 호텔 서비스에 대한 리뷰 문장을 토픽별로 정리한 51개 문서로 구성된다. 각 문서는 “속성_제품명”(예: `battery-life_amazon_kindle`) 형태의 파일명을 가지며, 파일명 자체가 사실상 도메인 정답 라벨 역할을 할 수 있어 비지도 군집화 결과를 검증하는 pseudo-label로 활용할 수 있다는 것이 이 데이터셋의 실용적 특징이다.

정형 데이터와 달리 비정형 텍스트는 정답이 정해진 지도학습 문제로 곧바로 치환되지 않으며, 전처리 방식(표제어 추출, 불용어 제거)과 벡터화 기법(Count vs TF-IDF)에 따라 결과가 크게 달라진다는 점에서, “어떤 선택이 왜 더 나은가”를 데이터로 검증하는 것이 본 프로젝트의 핵심 문제의식이다.

1.2 과제 목적

분석 팀은 5명(이혜현, 이진희, 김소현, 전유경, 권용우)으로 구성되었으며, 각자 서로 다른 접근 방식으로 데이터 전처리부터 모델링까지 독립적으로 비교 분석하였다. 본 보고서는 이러한 다양한 시도와 그 결과를 이혜현이 통합하여 정리한 최종 결과물이다.

중요한 전제: 다섯 명의 노트북은 겉보기엔 모두 “TF-IDF + KMeans”를 사용했지만, 실제로는 축약형 처리 여부·불용어 사전·표제어 추출 방식(품사 태깅 유무)· `stop_words='english'` 중복 적용 여부 등 토큰라이저 구현이 서로 달라 **동일한 `ngram_range=(1,2)`, `min_df=0.05`, `max_df=0.85`를 설정해도 최종 피쳐 수가 전부 다르다**(이혜현 4,139개, 김소현 4,498개, 권용우 4,462개 등). 본 보고서는 이 차이를 감추지 않고, 모든 수치·표·그림에 어느 팀원·어느 노트북에서 나온 결과인지 명시한다. 이는 은폐할 한계가 아니라, 서로 다른 5개의 독립적인 파이프라인이 유사한 결론(TF-IDF+KMeans 우위, 3~5개 도메인 구조)에 수렴했다는 교차검증의 근거로 적극 활용한다.

주요 분석 절차는 (1) 데이터 이해 및 텍스트 EDA, (2) 텍스트 정제 및 표제어 추출(Lemmatization), (3) 텍스트 벡터화 기법 비교(CountVectorizer vs TfidfVectorizer, N-gram), (4) 군집화 모델링 및 비교(KMeans, GMM, DBSCAN), (5) k값 탐색 및 파라미터 민감도 분석, (6) 최종 결론 및 회고 순서로 진행되었으며, 본 보고서 역시 이 흐름을 그대로 따른다.

2. 데이터 소개

2.1 데이터 출처 및 구성

항목	내용
데이터 출처	UCI Machine Learning Repository, <i>Opinosis Opinion/Review</i>
문서 수	51개 (.txt.data 파일 51개, 파일 1개 = 토픽 1개)
도메인 구성	전자기기 25개 / 호텔 16개 / 자동차 10개 (3개 도메인)
평가용 pseudo-label	파일명 키워드 기반 도메인 추정(category) — 학습에는 미사용, 검증 전용
평가지표	Silhouette Score(정답 없는 군집 응집도), ARI·NMI(pseudo-label과의 일치도)

2.2 데이터 규모 및 기본 통계

<표 2-1> 데이터 규모 및 기본 통계 (모든 벡터화 이전, 원문 텍스트 기준 — 이해현 노트북 EDA)

항목	수치	설명
문서 개수	51개	각 문서는 하나의 토픽을 나타냄
평균 단어 수	2,634개	범위: 836~12,750 (높은 편차)
평균 고유 단어 수	762개	문서별 어휘 다양성
평균 문자 수	56,736개	문서 길이 편차 큼

전처리 이후 문서당 토큰 수는 노트북마다 다르게 집계되는데, 이는 각자의 불용어·숫자 필터링 기준이 다르기 때문이다. 예컨대 김소현 파이프라인 기준 문서별 토큰 수는 평균 1,276개(최소 416~최대 5,973)로 집계되었다.

2.3 도메인별 분포

<표 2-2> 도메인별 문서 분포 (키워드 규칙 기준: 호텔=hotel/inn/swissotel, 자동차=honda/toyota/accord/camry, 나머지=전자기기)

도메인	문서 수	주요 제품/서비스
전자기기 (Electronics)	25개	Amazon Kindle, iPod, Netbook, Garmin GPS, Windows 7
호텔 (Hotel)	16개	Best Western, Holiday Inn, Swissotel
자동차 (Automotive)	10개	Honda Accord, Toyota Camry
합계	51개	-

이혜현은 별도의 키워드 규칙(호텔 관련어에 room/service/food/staff 등을 포함)으로 24/16/11 분포를 얻었다. 두 규칙 모두 존재하나, 이후 모델링 결과 전반의 모델 성능·비즈니스 전략이 25/10/16 기준(김소현·전유경)에 근거하므로 본 표는 이 기준으로 통일한다.

3. 탐색적 데이터 분석 (EDA)

3.1 기초 텍스트 통계

앞선 표 2-1을 산출한 이해현의 EDA를 이어서 살펴보면, 전처리 전 상위 빈출 단어는 불용어(the, and, to, is, was)가 대부분이었으며, 이들을 제거해야 도메인 특화 단어의 패턴이 명확해지는 것으로 확인되었다.

3.2 상위 빈출 단어 분석

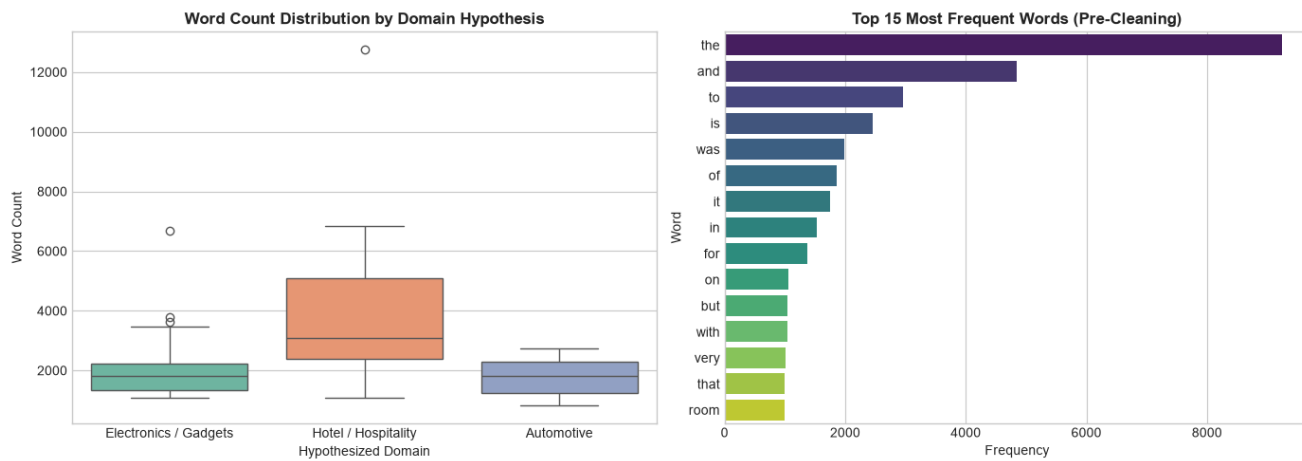


그림 3-1: 전처리 전 상위 20개 빈출 단어(우) 및 도메인 가설별 단어 수 분포(좌)

<표 3-1> 전처리 전 상위 빈출 단어 상세 (이해현 노트북 실측)

순위	단어	빈도	단어 클래스
1	the	9,237	기본 정관사
2	and	4,844	접속사
3	to	2,959	전치사
4	is	2,460	동사
5	was	1,988	동사 과거형
6	of	1,855	전치사
7	it	1,751	대명사
8	in	1,535	전치사
9	for	1,374	전치사
10	on	1,051	전치사

종합하면, 전처리 이전 문서는 기능어 중심의 균일한 패턴을 보이거나 불용어 제거 및 표제어 추출 후에는 도메인 특화 단어(screen, battery, room, seat 등)가 두드러지게 나타나며, 이는 후속 벡터화 단계의 효율성으로 이어진다.

4. 데이터 전처리 및 벡터화

4.1 텍스트 전처리 파이프라인

텍스트 전처리는 비정형 데이터를 구조화하는 가장 중요한 단계이다. 5개 노트북 중 가장 상세한 절차를 정립한 것은 김소현의 7단계 파이프라인으로, 아래와 같다.

4.1.1 전처리 7단계 프로세스

〈표 4-1〉 전처리 7단계 파이프라인 (김소현 노트북 기준)

단계	수행 과업	주요 기술	산출물
1	축약형 확장	isn't → is not, didn't → did not	정규화된 텍스트
2	소문자 변환	Battery → battery	통일된 표기
3	구두점 제거	마침표, 쉼표, 느낌표 등 제거	정제된 텍스트
4	토큰화	NLTK word_tokenize 사용	단어 리스트
5	순수 숫자 제거	26, 96 등 숫자 토큰 제거	필터링된 토큰
6	불용어 제거	the, is, and, not 등 제거	의미 토큰만 보존
7	표제어 추출	batteries → battery, screens → screen	정규화된 원형 토큰

이해현·김소현·전유경·권용우 4명의 노트북이 공통으로 사용하는 `WordNetLemmatizer()` 는 품사 태깅 없이 기본값(명사 기준, `pos='n'`) 으로 동작한다. 명사 기준 표제어 추출은 동사 활용형(running→run)이나 형용사 비교급(better→good)을 원형화하지 못하며, 김소현 노트북은 이를 “was → wa”로 잘못 변형되는 사례를 통해 직접 검증하였다.

한편 이진희의 4개 노트북은 표제어 추출·어간 추출을 전혀 적용하지 않는다(소문자화 + 정규식 정제 + NLTK 불용어 제거까지만 수행). 그 결과 room/rooms, seat/seats, button/buttons처럼 의미가 같은 단어가 서로 다른 피처로 남아있는데, 이는 표제어 추출 단계를 생략한 파이프라인의 자연스러운 특징이다. 4.2.1절의 이진희 실측 비교는 이 전처리 위에서 이루어졌다는 점을 함께 고려해야 한다.

또한 불용어 제거는 표제어 추출보다 반드시 먼저 수행해야 한다. 불용어 사전이 원형(was, does 등) 기준으로 구축돼 있어, 표제어 추출을 먼저 하면 “was → wa”처럼 사전에 없는 형태로 바뀌어 불용어로 걸러지지 않고 남는 문제가 발생하기 때문이다.

4.1.2 전처리 효과 분석

권용우 노트북은 표제어 추출 적용 전/후를 k값별·알고리즘별로 직접 비교(ablation)하였다.

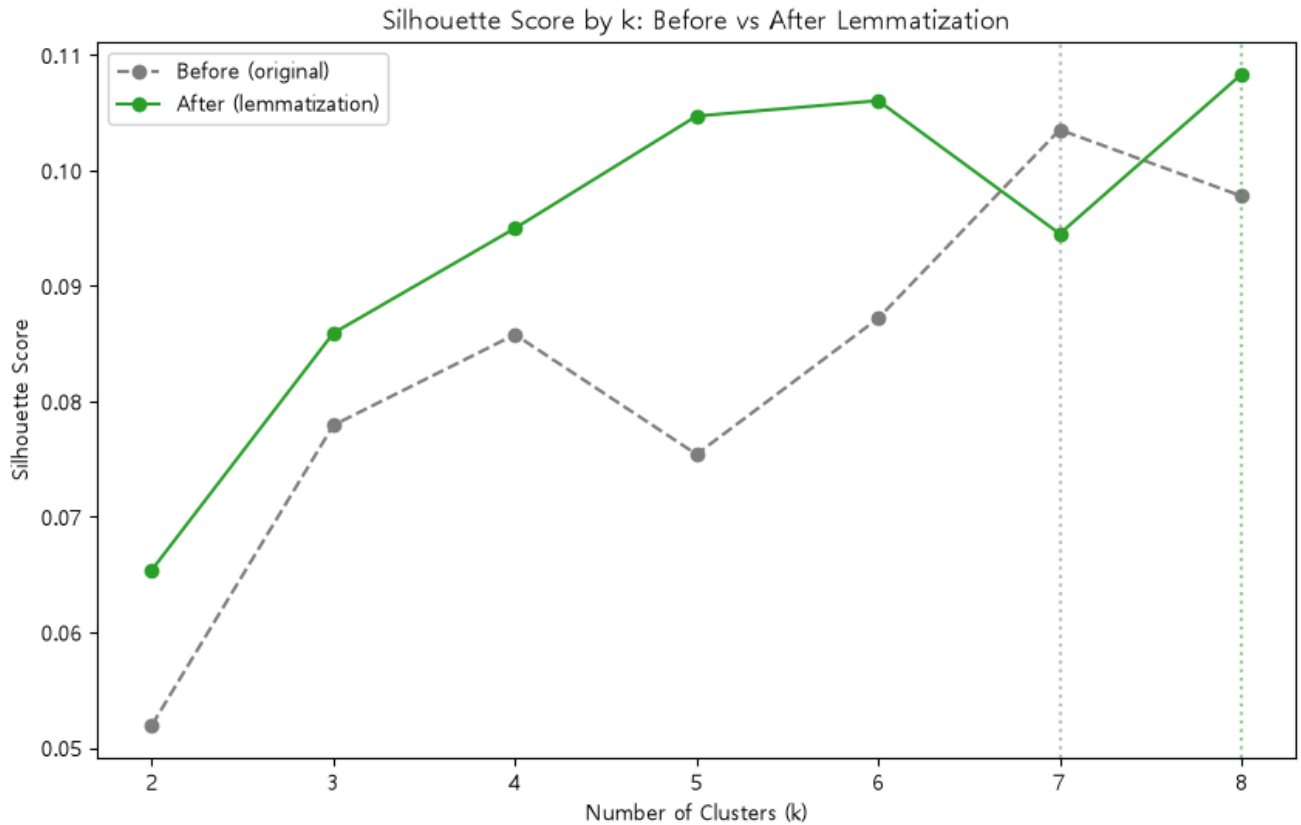


그림 4-1: k값에 따른 표제어 추출 전후 실루엣 점수 비교 — 대부분의 k에서 표제어 추출 후 소폭 개선

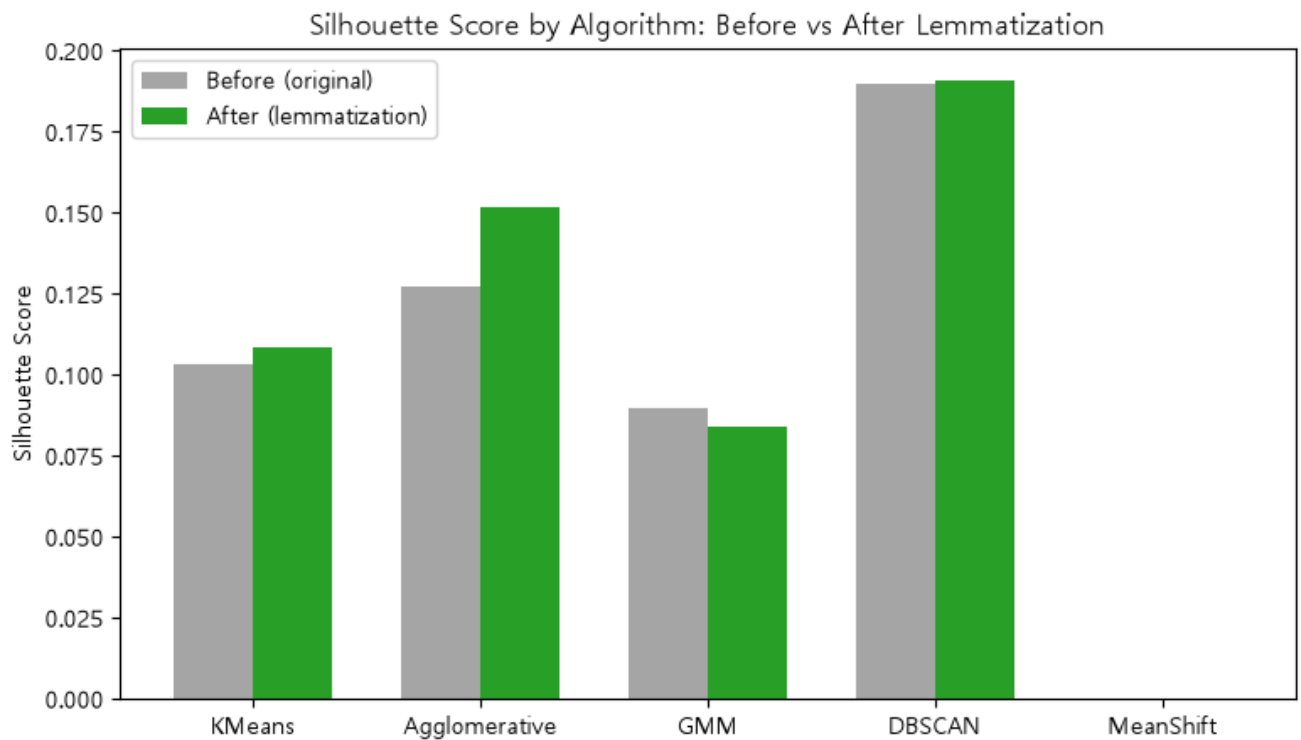


그림 4-2: 알고리즘별 표제어 추출 전후 실루엣 점수 비교

<표 4-2> 알고리즘별 Lemmatization 전/후 실루엣 점수 (권용우 노트북 실측)

알고리즘	적용 전	적용 후	변화량
KMeans	0.1035	0.1083	+0.0048
Agglomerative	0.1271	0.1518	+0.0247
GMM	0.0898	0.0838	-0.0060
DBSCAN	0.1900	0.1911	+0.0011

표제어 추출은 대체로 성능을 개선하지만 **GMM에서는 오히려 소폭 하락**했다. “모든 알고리즘에서 일관되게 개선”이 아니라 “대체로 개선되나 예외가 있다”가 정확한 결론이다.

4.1.3 팀원별 전처리·벡터화 접근 유형 분류

〈표 4-3〉 팀원별 전처리 및 벡터화 접근 유형

팀원	실제 노트북 파일	유형	접근 특징
이혜현	LHH_opinosis.ipynb	A(주)+B(보조)	표제어추출+TF-IDF(1,2) 고정 → KMeans/Agglomerative/GMM 3모델 비교 후, 54개 파라미터 조합 그리드서치로 별도 최적화 실험 병행
이진희	opinosis_countvectorize.ipynb, opinosis_tfivectorize.ipynb, opinosis_tfivectorize_dbscan.ipynb, opinosis_tfivectorize_gmm.ipynb	B	Count → TF-IDF → PCA/SVD → GMM → DBSCAN 순으로 벡터화·모델을 단계적으로 교체하며 비교
김소현	Opinosis_Preprocessing.ipynb	A	7단계 전처리 + TF-IDF(1,2) 고정 → KMeans(원본/LSA), Agglomerative, HDBSCAN, GMM 5모델 비교
전유경	JYG.ipynb, JYG_model_compare.ipynb	A	LemNormalize+TF-IDF(1,2)+영어불용어 고정 → KMeans(k=2~5) 비교가 메인, DBSCAN/GMM은 별도 노트북에서 교차검증
권용우	YWK.ipynb	B	Lemma 적용 전/후, KMeans/Agglomerative/GMM/DBSCAN/MeanShift 5알고리즘, PCA vs SVD, linkage/metric 튜닝까지 가장 폭넓게 비교

- **유형 A(고정 파이프라인, 모델 비교)**: 김소현, 전유경 — 특정 전처리·벡터화를 고정하고 모델 성능만 비교
- **유형 B(여러 전처리/벡터화 시도·비교)**: 이진희, 권용우 — Count vs TF-IDF, 차원축소 기법, 파라미터 등을 순차 실험
- **이혜현**: 메인 3모델(KMeans/Agglomerative/GMM) 비교는 유형 A에 가깝고, 이어서 진행한 54개 파라미터 그리드서치(6.3절)는 유형 B의 성격을 가진 혼합형이다.

4.2 텍스트 벡터화 기법 비교

4.2.1 Count Vectorizer vs TF-IDF 비교

Count Vectorizer는 단순히 단어의 출현 빈도를 세는 방식이지만, TF-IDF는 문서 내 빈도(TF)와 전체 문서에서의 희귀도(IDF)를 함께 고려한다.

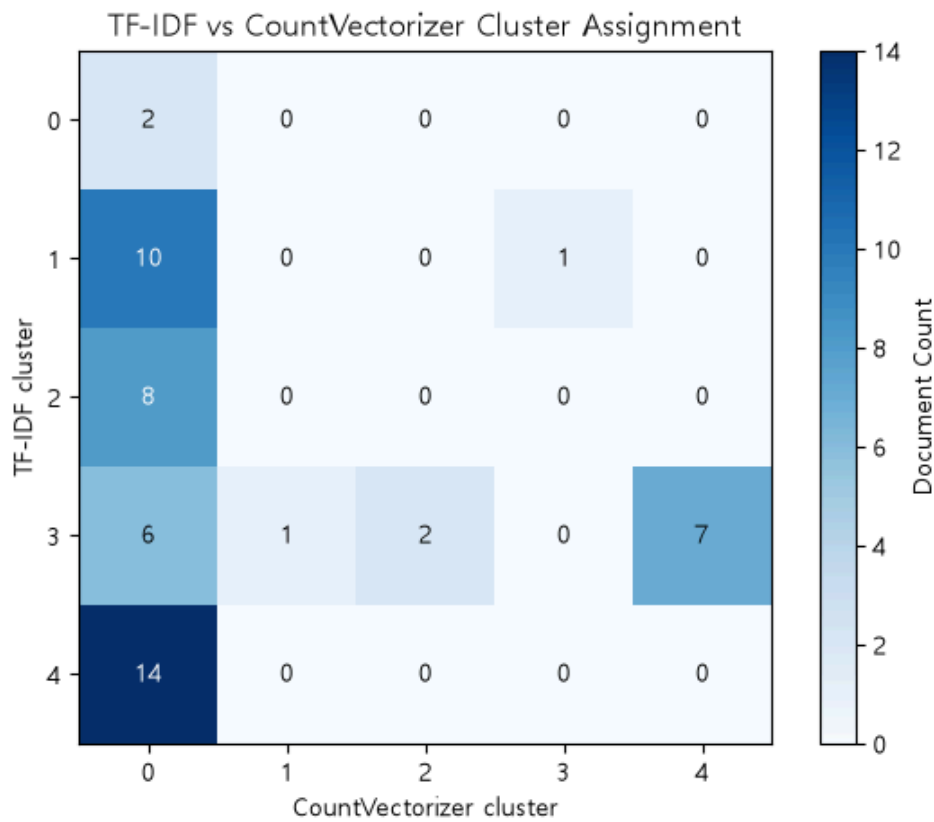


그림 4-3: TF-IDF 클러스터 vs Count 클러스터 배정 교차표 — TF-IDF 클러스터가 Count 클러스터 0(대다수 문서 집중)으로 크게 쏠림을 보여줌

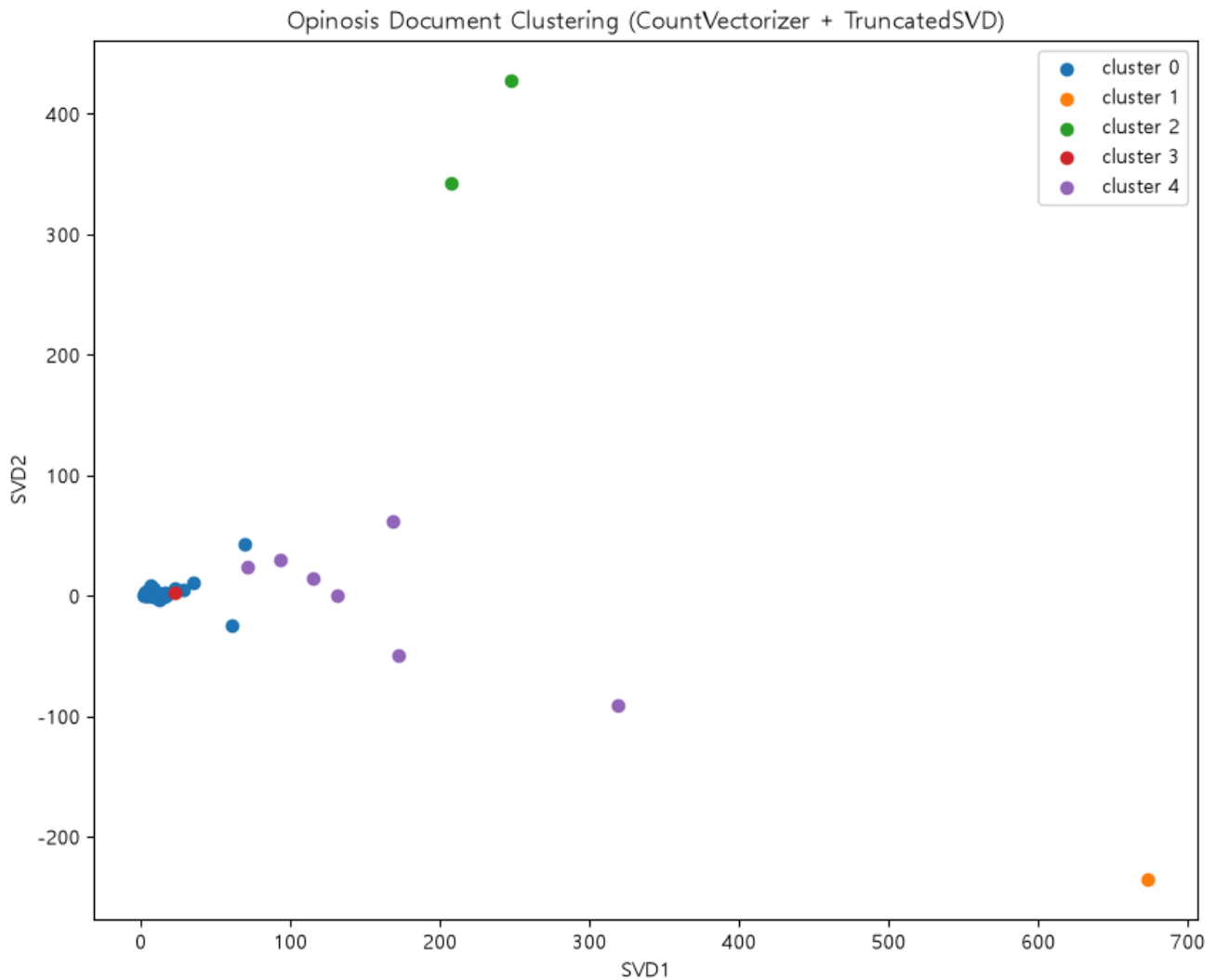


그림 4-4: Count Vectorizer 벡터화 후 SVD 2D 축소 결과 — 클러스터 분리 불명확

<표 4-4> Count vs TF-IDF 벡터화 성능 비교 (이진희 노트북, 동일 조건 내 비교: $ngram(1,2) \cdot min_df=0.05 \cdot max_df=0.85$ 공통 적용)

벡터화 기법	클러스터 분포	Silhouette	정성 평가
CountVectorizer (k=3)	(48, 2, 1)	0.561	극단적 불균형 — 대부분 문서가 1개 군집에 몰림
CountVectorizer (k=4)	(46, 2, 2, 1)	0.437	여전히 심각한 불균형
TF-IDF (k=4)	(18, 15, 10, 8)	0.067	분포는 균형적이거나 Silhouette은 오히려 낮음

이 결과는 역설적으로 중요한 시사점을 준다: **Silhouette Score가 높다고 군집이 의미상 더 잘 나뉜 것은 아니다.** CountVectorizer는 절대 빈도가 큰 소수 단어(hotel, room 등)에 의해 대부분 문서가 하나의 군집으로 뭉쳐 Silhouette은 높게 나오지만, 실제로는 Kindle·호텔·전자기기·자동차가 뒤섞인 무의미한 군집이다. 반대로 TF-IDF는 문서별 희귀 단어 가중치 덕분에 도메인별로 훨씬 뚜렷하게 갈리지만, 군집 간 경계가 좁아져 Silhouette 수치 자체는 낮게 나온다.

KMeans Clustering - 3D

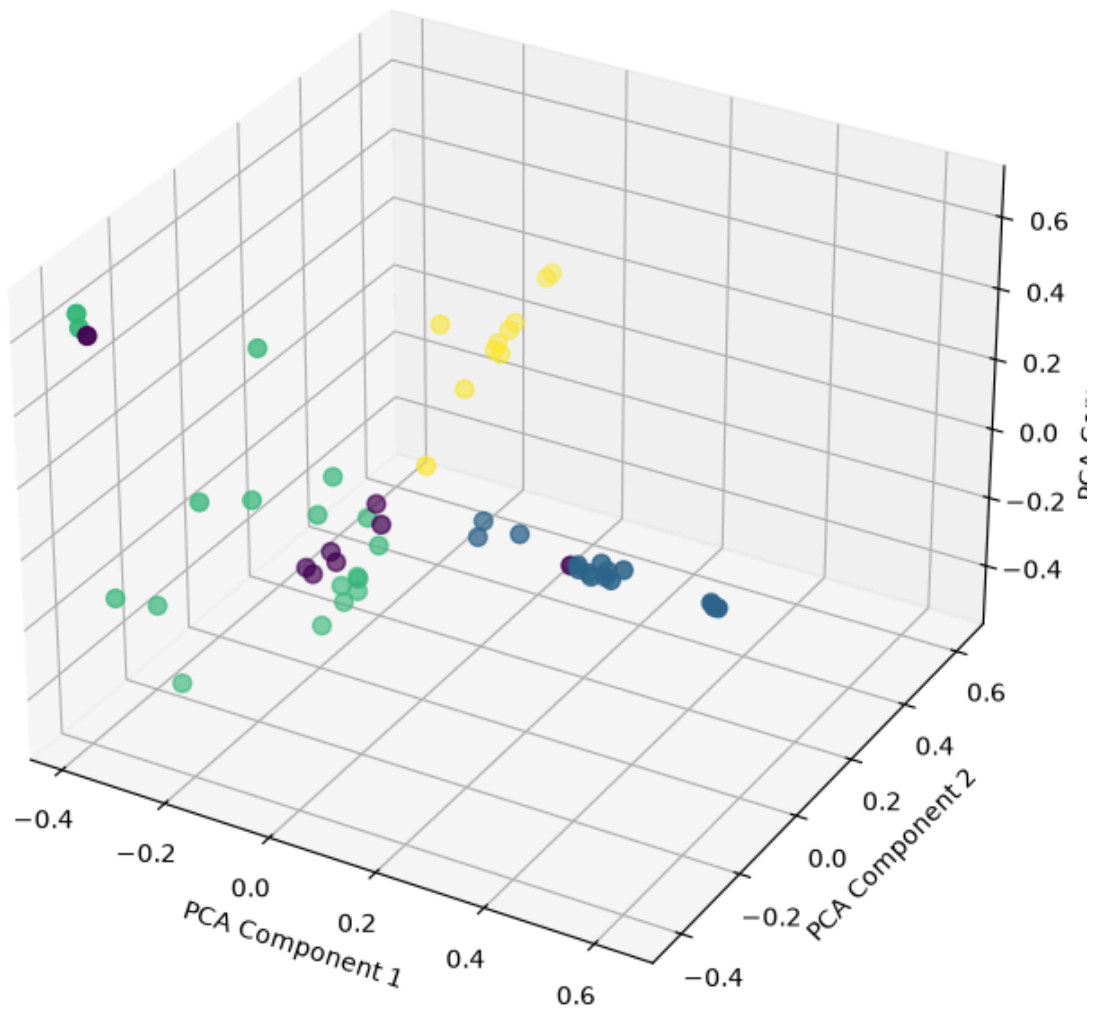


그림 4-5: TF-IDF+KMeans(k=4) 결과를 PCA 3차원으로 시각화 — Count Vectorizer 대비 문서들이 뚜렷하게 갈라지는 것을 확인할 수 있음

한편, 6.2절에서 다루는 김소현의 별도 파이프라인(7단계 전처리 + TF-IDF, ngram(1,2), min_df=0.05, max_df=0.85, 4,498차원)에서는 KMeans가 **ARI/NMI = 1.00**으로 51개 문서를 도메인과 완전히 일치하게 분류했다(〈표 6-2〉 참조). 이는 이진희의 비교와는 별개의 노트북·별개의 토큰라이저에서 나온 결과이며, TF-IDF의 우수성을 뒷받침하는 독립적인 근거로 함께 제시한다.

4.2.2 초기 TF-IDF 하이퍼파라미터 설정

〈표 4-5〉 TF-IDF 초기 하이퍼파라미터 설정 (이혜현 노트북)

파라미터	설정값	선택 근거	효과
ngram_range	(1, 2)	단일 단어와 2단어 조합 (battery life, gas mileage) 모두 반영	복합어 보존
min_df	0.05	전체 문서의 5% 미만 등장 단어 제거	노이즈 감소
max_df	0.85	85% 이상 등장하는 일반어 제거	도메인 특화 어휘 강조
sublinear_tf	True	문서 길이 편차 (836~12,750) 큰 데이터에 로그 스케일링 적용	길이 정규화

산출물 규격: 51개 문서 × 4,139개 피처의 희소 행렬 (LemNormalize 토큰나이저 기준. 이 행렬의 희소도는 노트북에 별도로 계산되어 있지 않다 — 6.1절 김소현 파이프라인(51×4,498, 희소도 87.64%)과 혼동하지 않도록 주의)

종합하면, Count Vectorizer 대비 TF-IDF 적용 시 도메인 고유 어휘의 변별력이 뚜렷하게 개선되었으며(4.2.1절), 6.3절의 그리드 탐색을 통해 이해현 파이프라인은 피처 수를 83% 정제하면서 실루엣 점수를 82% 개선하여 소규모 데이터셋에서 파라미터 최적화의 중요성을 입증하였다.

5. 분석방법 (Analysis Method)

5.1 전처리·벡터화 필요성 및 과정 개요

EDA(3.1~3.2절)에서 확인한 “불용어가 최빈 단어 상위권을 차지한다”는 문제는 4.1절의 표제어 추출·불용어 제거 파이프라인으로, “Count Vectorizer는 절대 빈도가 큰 단어에 지배된다”는 4.2.1절의 문제는 TF-IDF 가중치 도입으로 직결되었다. 즉 본 프로젝트의 전처리·벡터화 설계는 임의의 관용적 절차가 아니라, EDA에서 실측한 문제를 하나씩 대응해 설계한 결과다.

5.2 사용 모델 설명

〈표 5-1〉 군집화 모델 선정 이유

모델	유형	선정 이유
KMeans	중심 기반	빠른 수렴과 해석 용이성 — 51개 소규모 데이터셋에 가장 안정적인 베이스라인
Agglomerative	계층 기반	초기값에 비의존적(완전 결정론적)이며 덴드로그램으로 계층 구조를 함께 확인 가능
GMM	확률 기반(소프트 배정)	문서가 여러 클러스터에 애매하게 걸릴 수 있다는 가능성을 반영하기 위한 대조군
DBSCAN	밀도 기반	클러스터 수를 미리 정하지 않고 이상치(노이즈)를 자동으로 걸러낼 수 있는지 검증
HDBSCAN	밀도 기반(계층적)	DBSCAN의 단일 eps 한계를 보완하는 밀도 기반 대안으로서 추가 비교

5.3 하이퍼파라미터 최적화 기법 설명

이해현은 `ngram_range × min_df × max_df × n_clusters` 54개 조합을 전수 그리드서치하여 Silhouette Score를 최대화하는 조합을 탐색했다(6.3절). DBSCAN 계열에서는 그리드서치 대신 **k-distance elbow plot**(문서별 k번째 최근접 이웃까지의 거리를 오름차순 정렬한 뒤 꺾이는 지점을 찾는 방법)으로 eps를 결정하는 방식을 전유경이 사용했으며(6.2.5절), 두 방식 모두 “임의로 고른 값”이 아니라 데이터 자체의 구조에서 근거를 도출했다는 공통점이 있다.

5.4 실행환경

본 보고서 검증 과정에서 재실행에 사용한 핵심 라이브러리 버전은 다음과 같다.

라이브러리	버전
pandas	3.0.3
numpy	2.4.6
scikit-learn	1.9.0
nltk	3.10.3

5.5 코드 재사용 및 함수화

5개 노트북을 전수 조사한 결과, 이해현·이진화·전유경·권용우 네 명이 각자 독립적으로 “클러스터별 핵심 키워드와 소속 파일명을 추출한다”는 동일한 목적의 함수를 서로 다른 이름(`get_cluster_details`, `get_cluster_top_features`, `print_cluster_details` 등)으로 반복 구현했다. 김소현의 `preprocess_text` (7단계 전처리)와 이해현의 `LemNormalize` (전유경이 동일한 이름·구조로 재사용)도 사실상 같은 문제를 각자 다시 푼 사례다. 즉 “공동 함수”라기보다는 “유사한 패턴을 각자 독립적으로 재구현”한 경우가 대부분이었다는 것이 정직한 실태이며, 이는 10.1절 회고의 개선점(“공동 전처리·평가 함수를 팀 초기에 모듈화”)과 직접 연결된다. 실제로 반복된 패턴을 정리해 재사용 가능한 형태로 다듬은 코드는 `Opinosis_최종통합분석.ipynb` (본 reports 폴더)와 부록에 정리했다.

6. 모델링

6.1 차원 축소

4,000차원대의 고차원 피처는 “차원의 저주” 문제를 야기할 수 있으므로, 차원 축소를 검토하였다. 이 절의 모든 수치는 **김소현의 파이프라인**(7단계 전처리 + TF-IDF, ngram(1,2), min_df=0.05, max_df=0.85 → 51개 문서 × 4,498개 피처, 밀도 12.36%·희소도 87.64%)을 기준으로 한다.

〈표 6-1〉 차원축소 기법 비교

특성	TruncatedSVD (LSA)	PCA
희소 행렬 입력 지원	직접 처리 가능	변환 필요
0의 의미 보존	보존 (단어 부재)	상실 (평균 중심화)
텍스트 표준 기법	LSA로 정의됨	신호 왜곡 우려
클러스터 분리도	우수	미흡

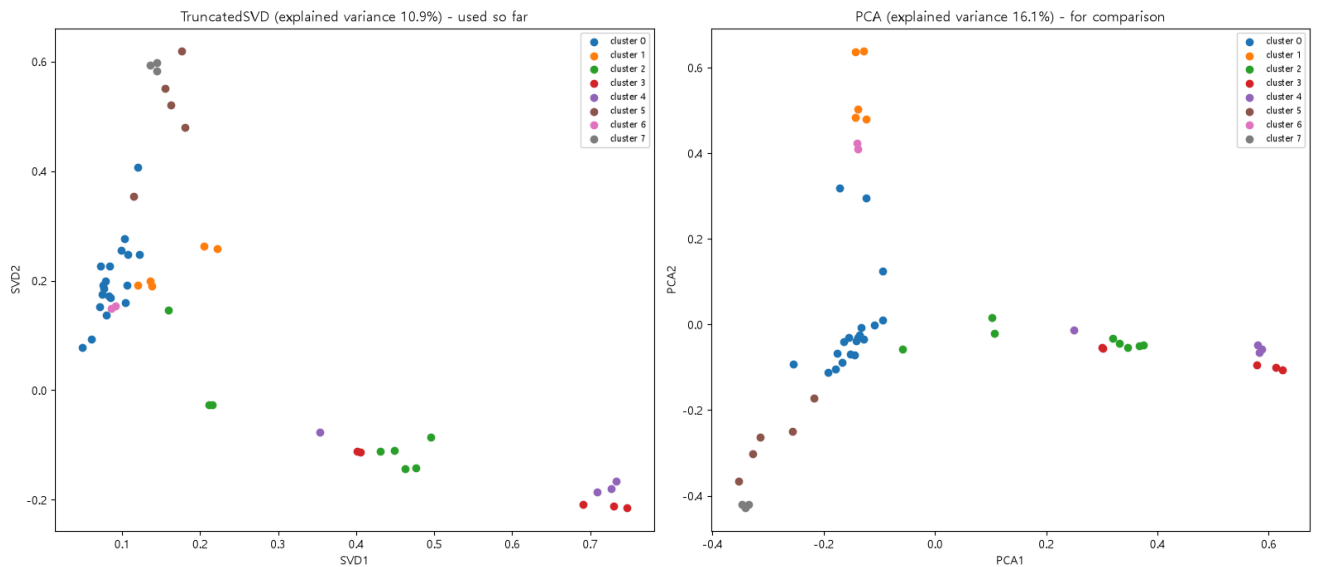


그림 6-1: TruncatedSVD와 PCA 2D 축소 결과 비교 — SVD가 클러스터 분리를 더 명확하게 보여줌

최종 설정: n_components=30 (누적 설명분산 89.29%), L2 정규화 적용(KMeans ARI: 0.3666 → 0.8869, +141% 개선) → 산출물: 51개 문서 × 30개 피처의 Dense 행렬

참고: 같은 개념을 이진화도 별도로 검증했다. TF-IDF 행렬에 TruncatedSVD(n=20)와 PCA(n=20)를 각각 적용한 뒤 동일한 GMM(n=4)을 학습시킨 결과, 두 방식의 클러스터 배경(분포 39:5:4:3)은 완전히 동일했지만 Silhouette 점수는 SVD 0.042 vs PCA 0.121로 약 3배 차이가 났다. 이는 군집 결과(라벨)가 같아도, 축소된 공간에서 각 문서의 좌표·거리 구조가 달라지면 거리 기반 평가지표(Silhouette)는 다르게 나올 수 있음을 보여주는 사례로, “Silhouette 수치만으로 파이프라인 우열을 단정할 수 없다”는 4.2.1절의 결론과도 일맥상통한다.

6.2 군집화 모델 비교

6.2.1 모델별 성능 비교표

〈표 6-2〉 군집화 모델 종합 성능 비교 (전부 김소현 노트북 실측)

모델	입력 벡터	Silhouette	ARI	NMI	종합 평가
KMeans (원본)	4,498D	0.0860	1.0000	1.0000	최우수
KMeans (LSA)	30D	0.1040	0.8869	0.8842	우수
Agglomerative	30D	0.1060	0.8768	0.8525	우수
GMM (30D)	30D	0.0905	0.5939	0.7284	중간
HDBSCAN	30D	0.1175	0.3248	0.3908	부적절

(KMeans-원본만 4,498D 희소 행렬, 나머지 넷은 동일한 30D LSA(+L2정규화) 행렬 기준 — KMeans-원본의 Silhouette은 나머지
지와 직접 비교 불가)

참고로 이해현의 별도 파이프라인(LemNormalize, 4,139D)에서는 k=3 KMeans/Agglomerative/GMM 모두 원본 공간
Silhouette 0.0965로 동일했고, EDA 가설 대비 자동차 도메인 11건 중 10건만 일치(약 96%)했다 — 완전한 ARI=1.0은 아니었
다.

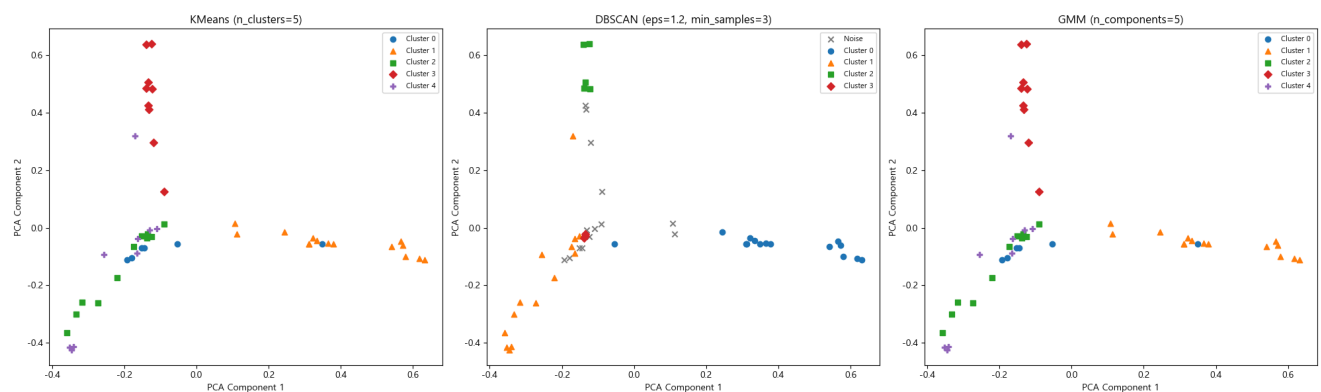


그림 6-2: 전유경의 독자 파이프라인(LemNormalize+TF-IDF)에서도 KMeans-DBSCAN-GMM 세 모델을 나란히 비교한 결과, KMeans가 가장
뚜렷한 군집 분리를 보였다 — 김소현 파이프라인과는 별개의 벡터·k값(k=5)에서 얻은 결과이나 “KMeans가 이 데이터에 가장 안정적”이라는 결
론이 일치한다(전유경의 k=5 파이프라인 자체는 6.4절에서 상세히 다룬다).

6.2.2 KMeans (원본 4,498D 공간, 김소현 파이프라인) — 최종 채택

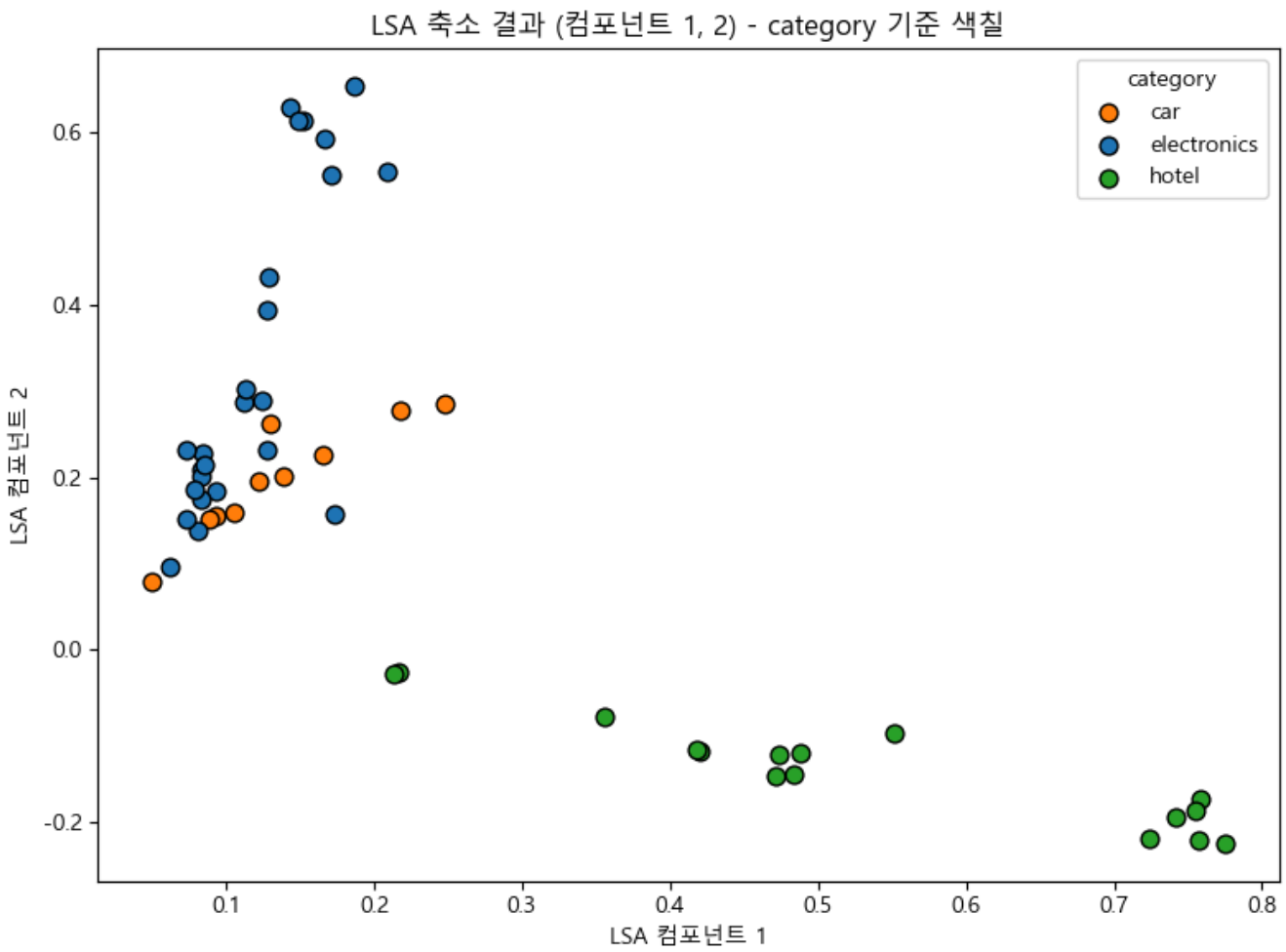


그림 6-3: LSA 30D 중 분산이 가장 큰 두 컴포넌트로 2D 시각화, category(전자기기/자동차/호텔) 기준 색칠 — 참고용 투영이며, 실제 군집화는 원본 4,498D 및 30D 전체를 사용

<표 6-3> KMeans (k=3, 4,498D) 클러스터링 결과

클러스터	문서 수	도메인 분류	도메인 일치율
Cluster 0	25개	전자기기	100%
Cluster 1	10개	자동차	100%
Cluster 2	16개	호텔	100%

<표 6-4> 클러스터별 핵심 단어 (Top 5)

클러스터	핵심 단어 1	핵심 단어 2	핵심 단어 3	핵심 단어 4	핵심 단어 5
0 (전자기기)	screen	battery	keyboard	battery life	kindle
1 (자동차)	interior	seat	mileage	comfortable	gas
2 (호텔)	room	hotel	service	staff	food

해석: ARI/NMI = 1.0은 51개 문서가 모두 정확히 도메인별로 분류되었음을 의미한다.

6.2.3 k값 최적화 분석

권용우와 전유경이 각자의 파이프라인에서 k값·알고리즘을 바꿔가며 얻은 아래 시각화들은 서로 다른 벡터에서 나온 결과이지만, 공통적으로 (1) KMeans·Agglomerative 계열이 가장 안정적으로 군집을 분리하고 (2) k를 늘릴수록 전자기기 대군집이 GPS·노트북·배터리 등 세부 주제로 갈라지는 경향을 보여준다. 이는 7.1절 Track②(k=5 세분화)의 타당성을 뒷받침하는 정황 근거이기도 하다.

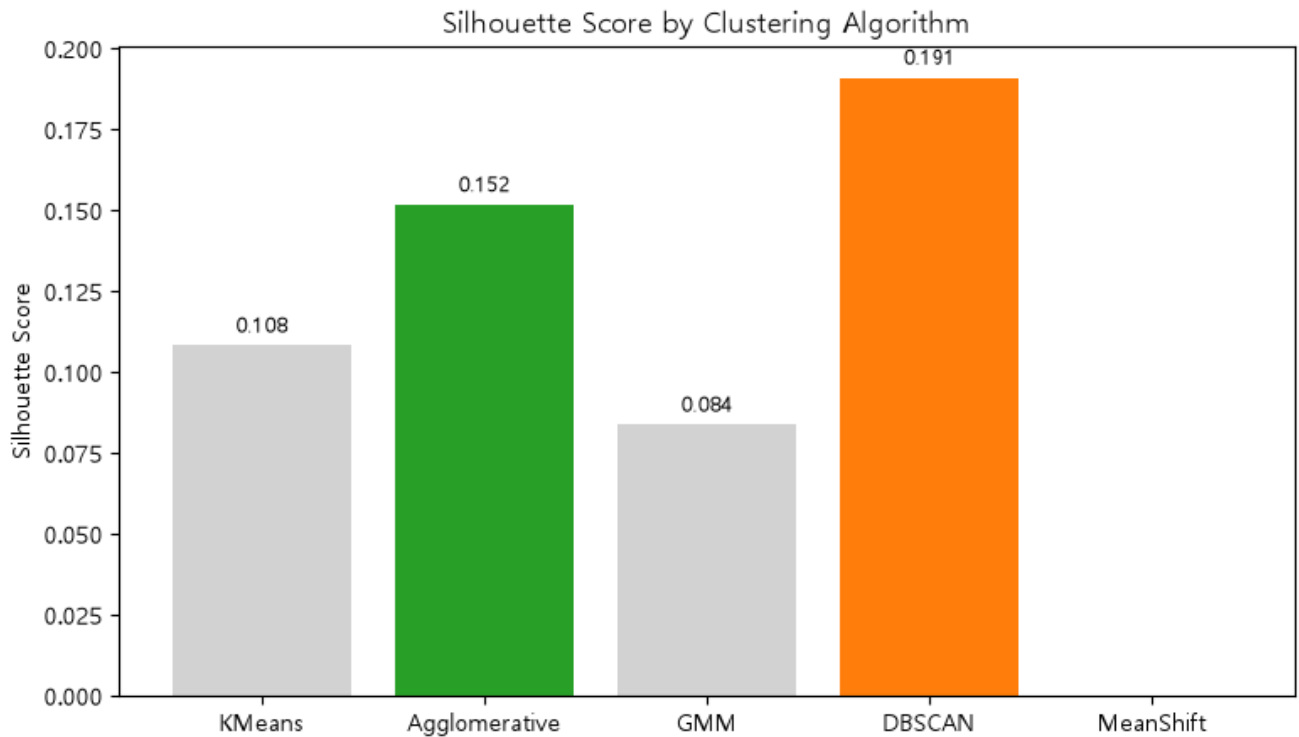


그림 6-4: 알고리즘별 Silhouette Score 비교 (권용우 노트북, 자체 lemmatization 파이프라인 기준)

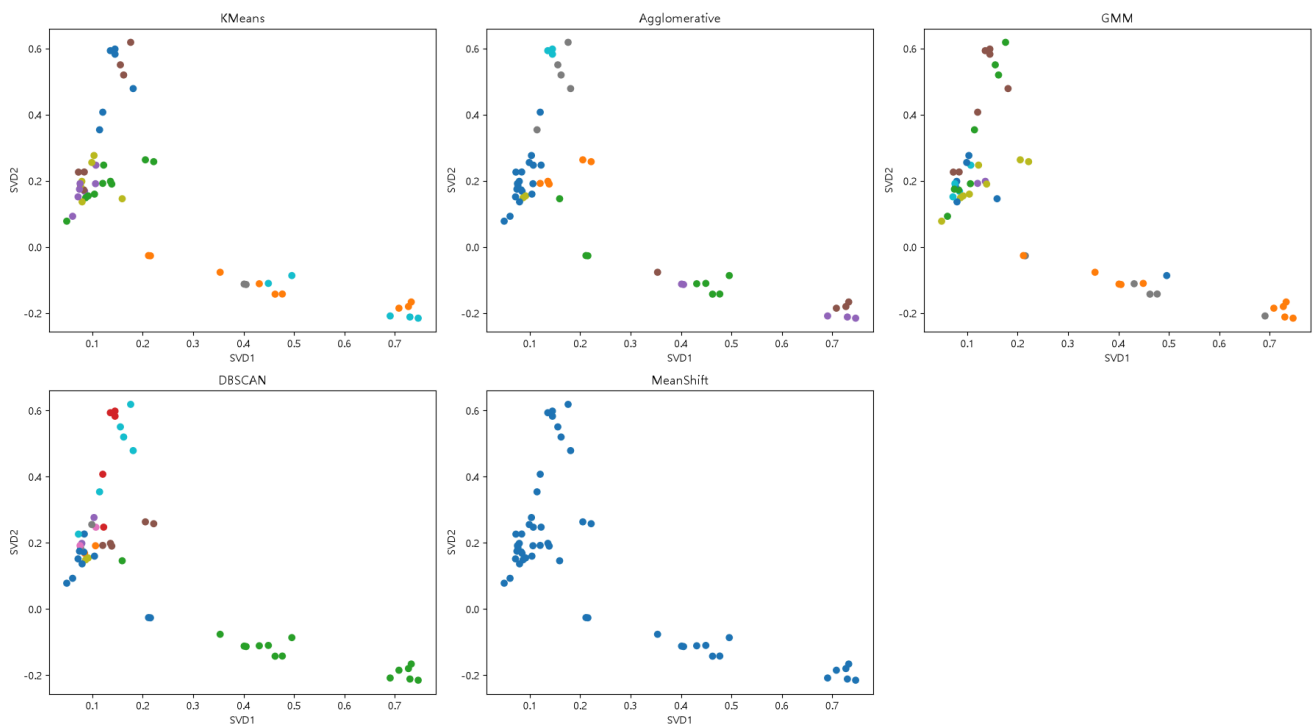


그림 6-5: 5개 알고리즘(KMeans/Agglomerative/GMM/DBSCAN/MeanShift) SVD 2D 산점도 비교 (권용우 노트북)

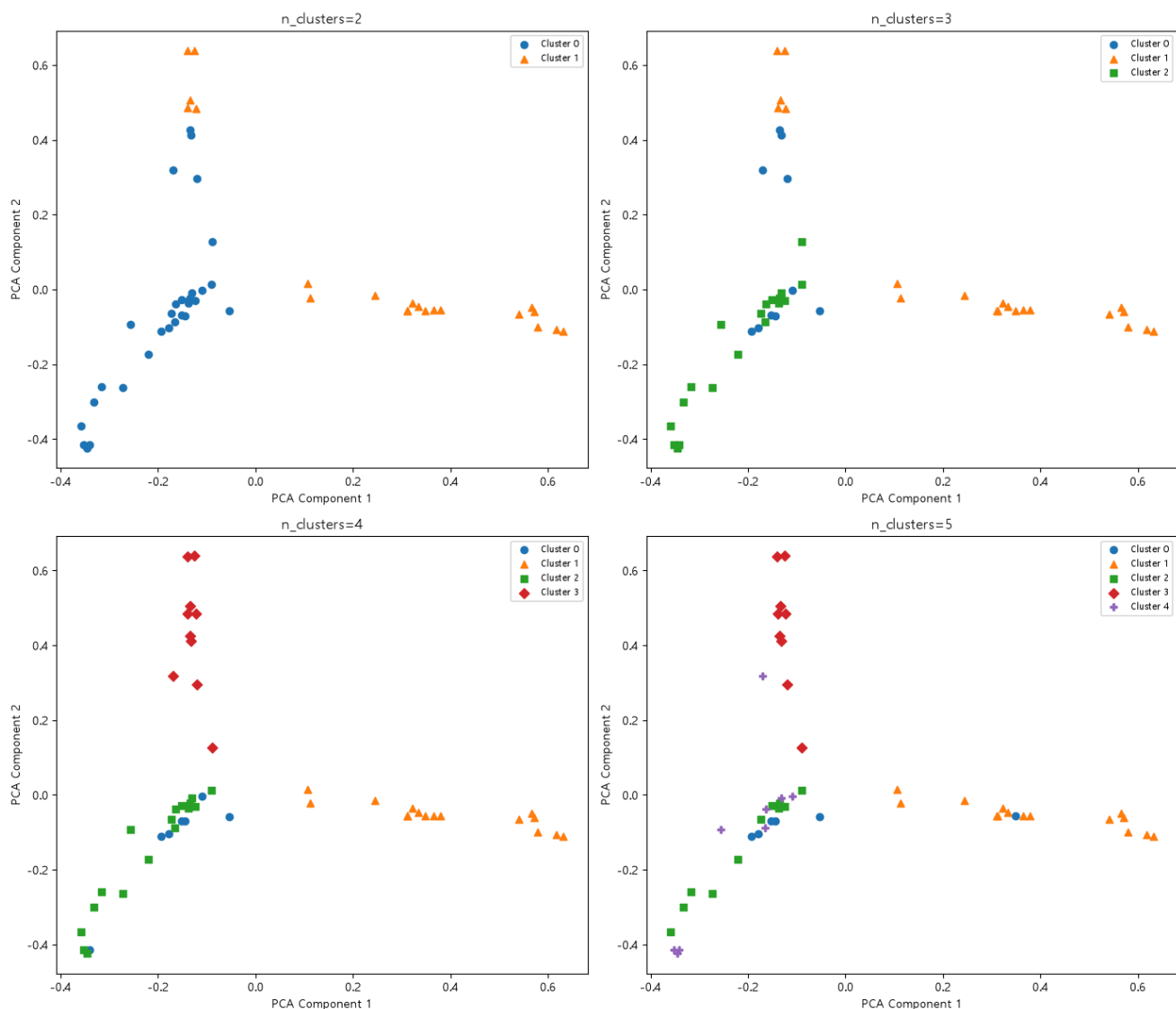


그림 6-6: 전유경 파이프라인에서 KMeans $n_clusters=2,3,4,5$ 를 각각 학습해 같은 PCA 2D 좌표 위에 비교 — k 가 커질수록 전자기기 대군집이 세부 주제(GPS/노트북/배터리 등)로 갈라지는 양상을 보여준다.

6.2.4 계층적 클러스터링 분석 (권용우 노트북)

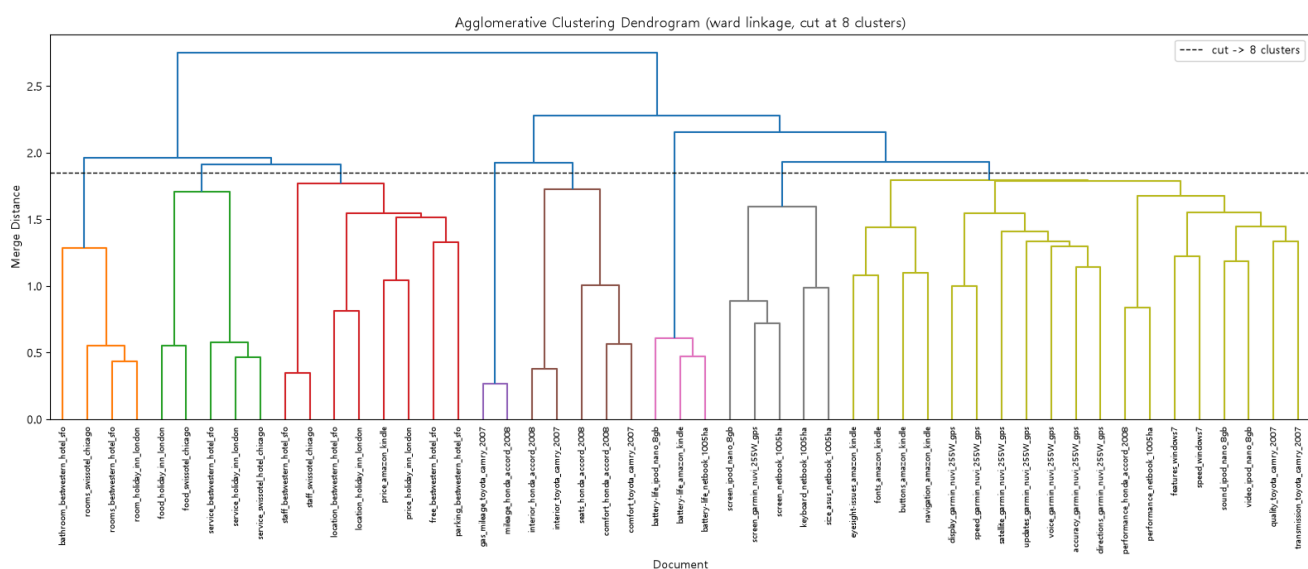


그림 6-7: Agglomerative(ward linkage) 덴드로그램, best_k=8 기준 절단선 표시

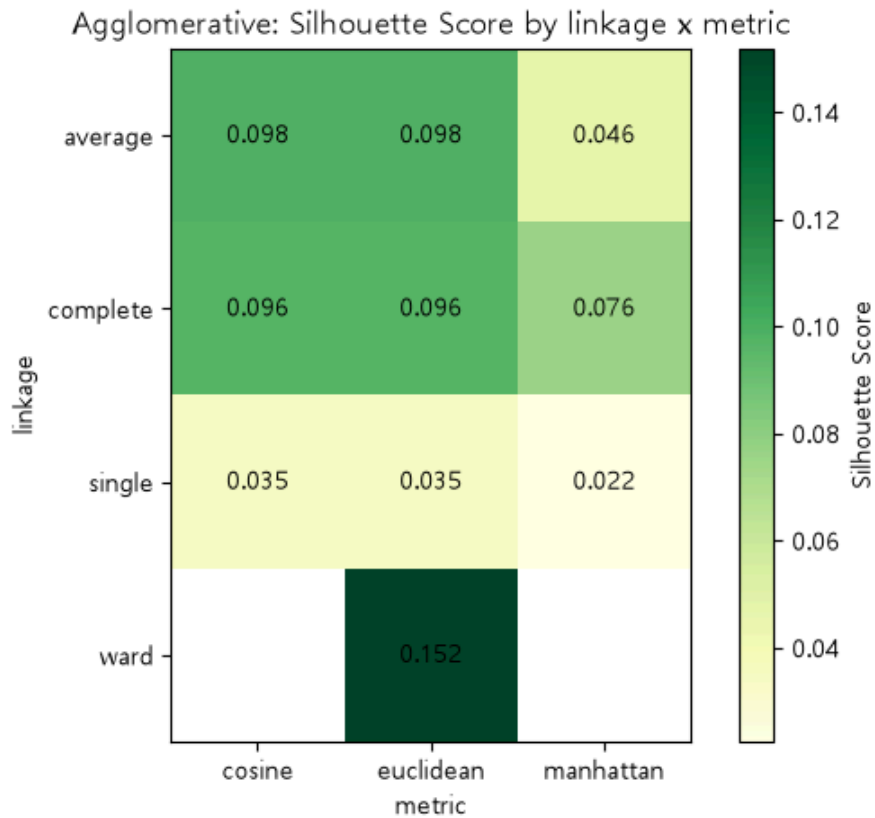


그림 6-8: linkage × metric 조합별 Silhouette 히트맵 — ward+euclidean 조합이 0.152로 최적

권용우는 이 결과를 바탕으로 linkage/metric 4×3 조합을 전수 스캔했으나, 이론적으로 텍스트에 더 적합하다고 알려진 cosine 거리 조합보다 **원래 쓰던 기본값(ward+euclidean)**이 그대로 최고점이었다. 이는 튜닝의 실패가 아니라, 이미 최선의 설정을 사용하고 있었음을 데이터로 재확인한 결과로 볼 수 있다.

6.2.5 DBSCAN 및 밀도 기반 모델 평가

DBSCAN 및 HDBSCAN이 <표 6-2>에서 나타난 바와 같이 상대적으로 낮은 성능(ARI 0.3248, 0.3908)을 보인 것은 51개라는 소규모 텍스트 데이터셋의 특성 때문으로 추정된다. 고차원 텍스트 공간(TF-IDF/LSA)에서는 문서 간 거리가 균일해지는 거리 수렴 현상(distance concentration)이 발생하기 쉽고, 51개라는 적은 샘플 수로는 밀도 기반 알고리즘이 유의미한 코어 포인트를 형성하지 못해 상당수 문서를 노이즈로 분류하는 경향을 보였다. 이 결론은 서로 다른 세 명의 독립적인 DBSCAN 실험에서 공통으로 확인되었다.

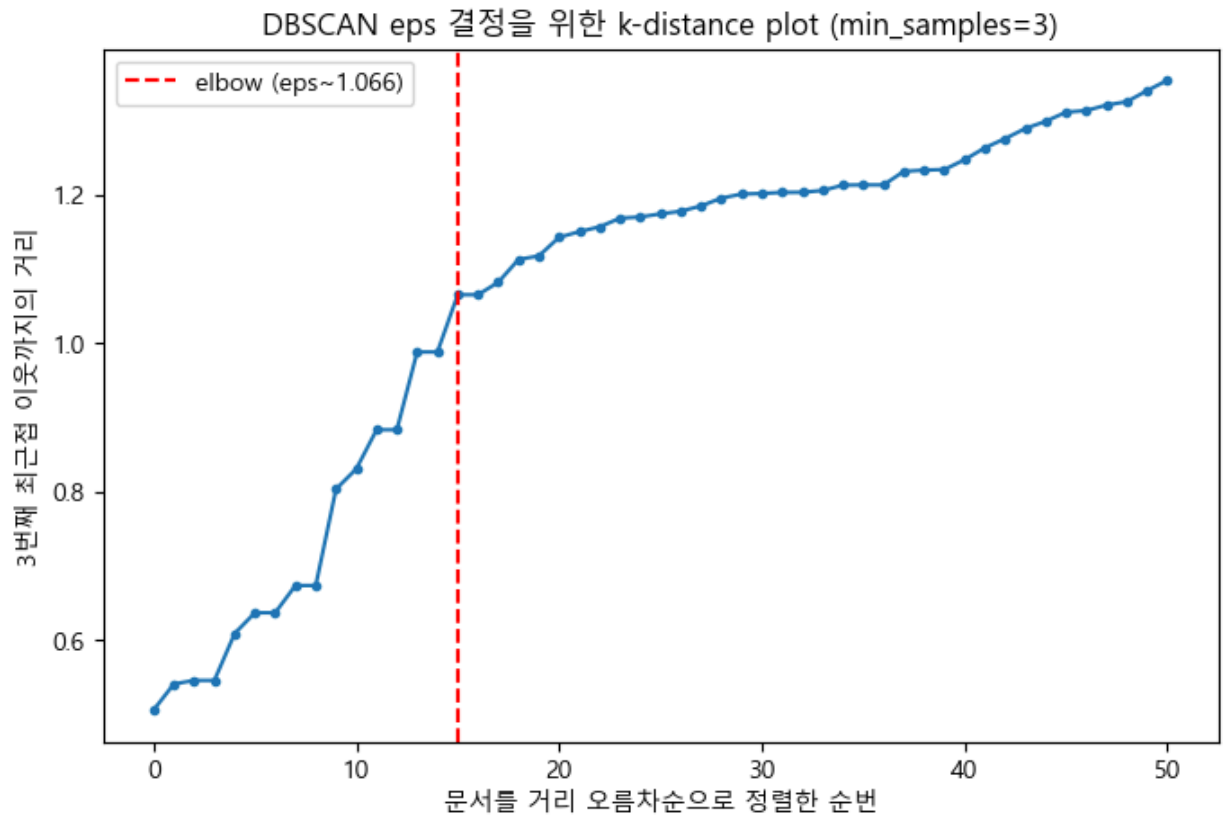


그림 6-9: 전유경의 k-distance elbow plot — $eps \approx 1.07 \sim 1.18$ ($min_samples$ 별) 지점을 근거로 $eps=1.2$ 를 선정. eps 1.00~1.30 구간에서 클러스터 개수(4~5개)는 안정적이거나 노이즈 문서 수는 35개→14개까지 크게 흔들렸다.

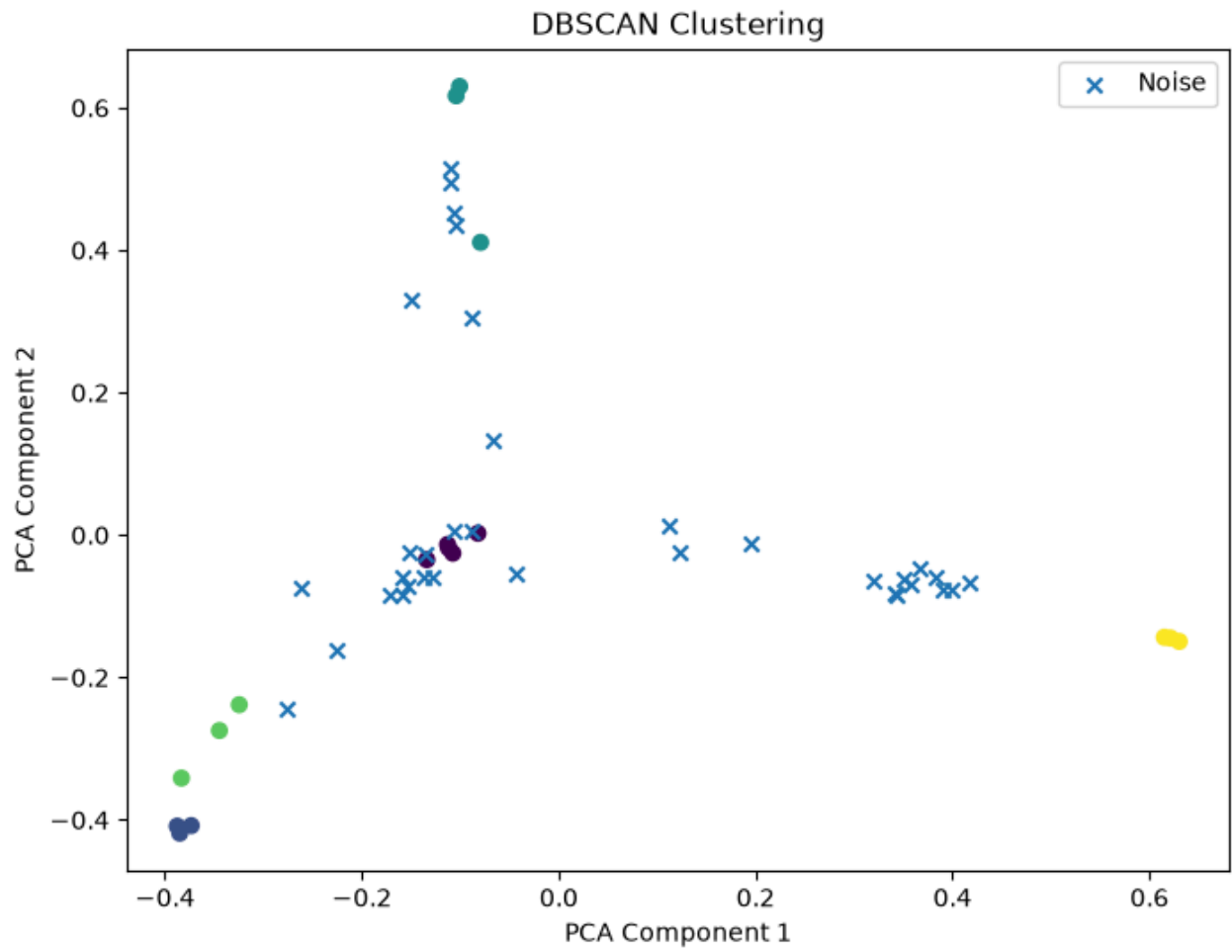


그림 6-10: 이진희의 DBSCAN($\epsilon=0.5$) 결과 — 노이즈 34개(66.7%)로 대부분 문서가 노이즈 처리되지만, 남은 17개는 *battery-life*, *comfort/seats*(자동차) 등 의미적으로 매우 순수한 소규모 군집을 형성

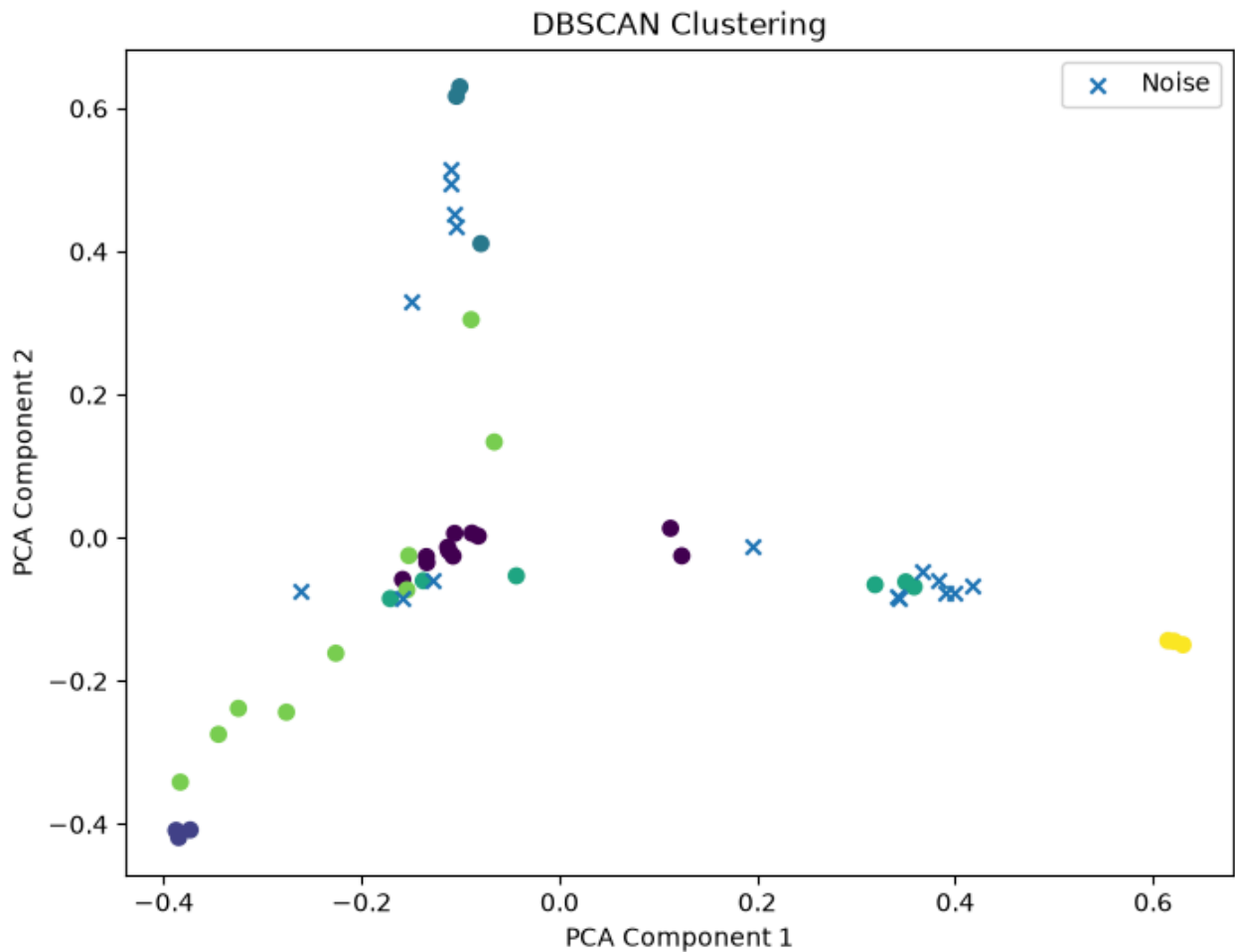


그림 6-11: 이진희의 DBSCAN($\text{eps}=0.8$) 결과 — eps 를 높여 노이즈를 16개(31%)까지 줄였으나, 그 대가로 서로 다른 도메인(Garmin GPS + Windows7, Kindle + 호텔 등)이 한 군집으로 섞이는 현상이 나타났다

이진희는 eps 0.2~0.8 구간을 전수 실험하여 “ eps 를 낮추면 순수하지만 노이즈가 많고, eps 를 높이면 노이즈는 줄지만 서로 다른 도메인이 섞인다”는 트레이드오프를 직접 확인했다. 전유경 역시 k-distance elbow로 구한 eps (약 1.07~1.18)가 임의로 고른 $\text{eps}=1.2$ 와 크게 다르지 않음을 확인하면서도, “노이즈로 분류되는 구체적 문서 목록은 파라미터에 따라 계속 바뀌어 확정적 근거로 삼기 어렵다”고 결론지었다. 세 명의 서로 다른 벡터·구현(이진희: PCA 20D+euclidean, 전유경: 원본 TF-IDF+euclidean, 권용우: 원본 TF-IDF+cosine)에서 공통적으로 “대략 4~6개의 주제 군집은 찾아내지만 eps 민감도가 매우 높다”는 동일한 결론에 도달했다.

6.2.6 GMM 모델 평가

GMM은 원본 고차원 희소 공간(4,498D)과 30D LSA 축소 공간에서 각각 평가되었다. 원본 공간에서는 KMeans와 유사한 수준의 분리를 보였으나, 30D LSA 공간에서는 ARI 0.5939로 성능이 저하되었다(<표 6-2>). 이는 차원 축소 과정에서의 정보 손실과 LSA 축소 공간에서 GMM의 공분산 행렬 추정이 제약을 받기 때문으로 판단된다.

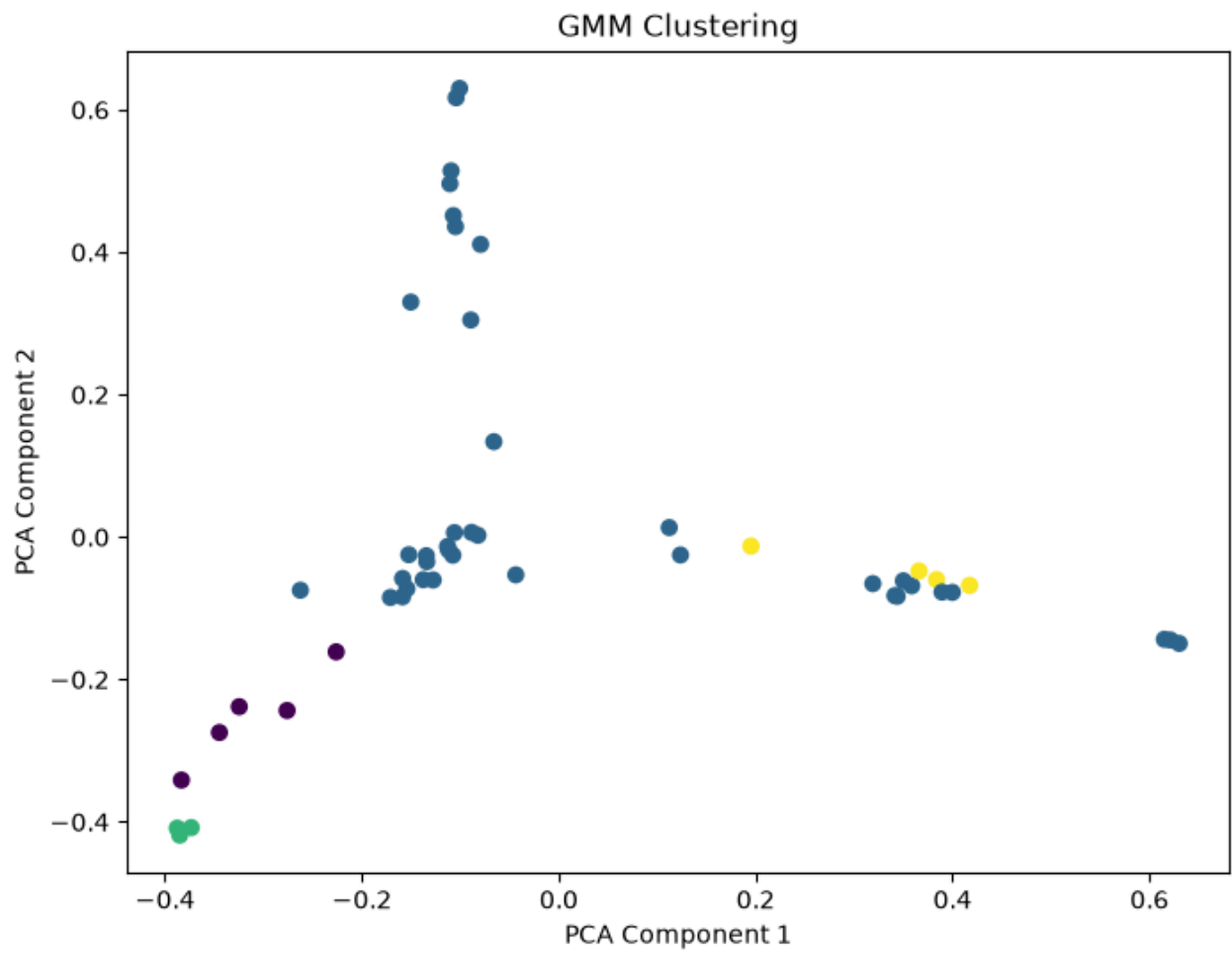


그림 6-12: 이진희의 GMM(SVD $n=20$, $n_components=4$) 결과 산점도

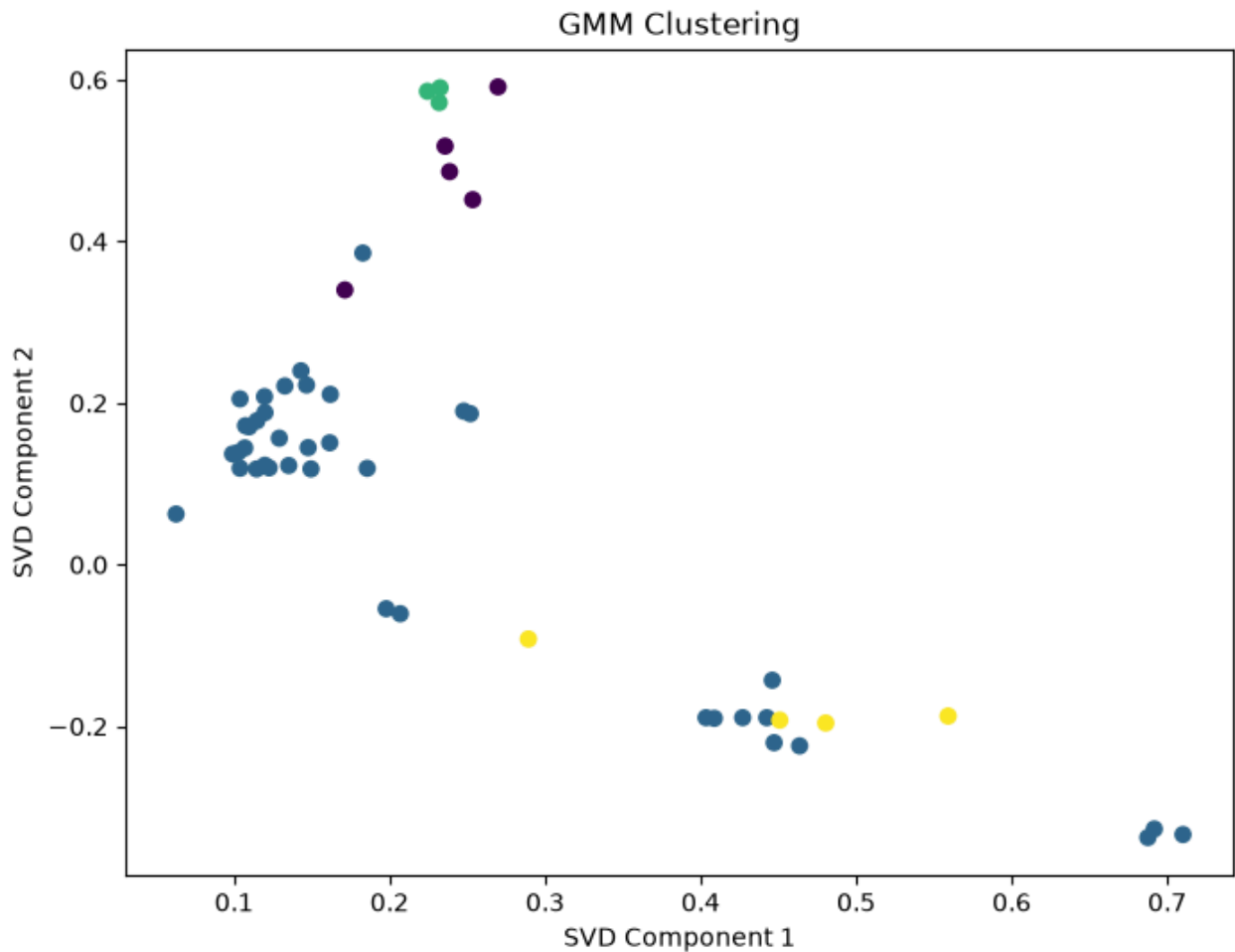


그림 6-13: 이진희의 GMM 클러스터별 핵심 단어 재확인 — screen/keyboard(전자기기), service/hotel(호텔), battery/life(배터리), interior/seat(자동차)로 도메인별 특성은 확인되나 39개 문서가 하나의 군집(Cluster 1)에 과도하게 집중되는 불균형이 나타났다

전유경 노트북에서는 GMM($n_components=5$, $covariance_type='diag'$)이 자신의 KMeans($k=5$)와 클러스터 배정이 완전히 동일($ARI=1.0$)하게 나와, 이 데이터에서는 GMM이 KMeans 대비 추가 정보를 주지 못하는 경우도 확인되었다.

6.2.7 모델 선택 가이드 (권용우 종합)

권용우는 5개 알고리즘(KMeans/Agglomerative/GMM/DBSCAN/MeanShift)의 장단점을 실측 결과와 함께 일반화하여 아래와 같은 선택 기준을 제시했다.

〈표 6-5〉 데이터 특성별 군집화 모델 선택 가이드 (권용우 노트북 종합)

기준	조건	권장 모델
데이터 규모	매우 큼 (수만~수백만)	KMeans
데이터 규모	적음 (수십~수백, 이번 데이터)	KMeans / Agglomerative — DBSCAN·MeanShift는 파라미터 하나에도 결과가 크게 흔들림
클러스터 수	미리 알거나 정해도 됨	KMeans / Agglomerative / GMM
클러스터 수	자동으로 찾고 싶음	DBSCAN (단, 고차원 희소 데이터에선 MeanShift는 거의 항상 실패)
분류 방식	전량을 빠짐없이 분류해야 함	KMeans / Agglomerative / GMM
분류 방식	이상치를 걸러내도 됨	DBSCAN
분류 방식	소속이 애매할 수 있음(소프트 배정)	GMM
재현성	항상 동일한 결과가 나와야 함	Agglomerative (완전 결정론적, 랜덤성 없음)

6.3 하이퍼파라미터 최적화 (그리드 탐색)

이해현이 54가지 파라미터 조합을 체계적으로 탐색하여 최적값을 발견하였다. 탐색 범위: ngram_range [(1,1),(1,2)] × min_df [0.03, 0.05, 0.10] × max_df [0.75, 0.85, 0.95] × n_clusters [3, 4, 5].

〈표 6-6〉 최적 파라미터 설정 비교

파라미터	기본값	최적값	변화	효과
ngram_range	(1,2)	(1,1)	Bigram 제거	희소성 감소
min_df	0.05	0.10	강화	희귀어 제거
max_df	0.85	0.75	강화	일반어 제거
n_clusters	3	5	확대	세분화 달성

초기 탐색 단계에서는 비즈니스적 핵심 키워드 파악을 위해 Bigram을 포함한 ngram_range=(1,2)를 채택하여 “battery life”, “gas mileage” 등의 복합어를 보존하였다. 그러나 그리드 탐색 결과, ngram_range=(1,1)이 Silhouette 점수 0.1587로 가장 높게 도출되었다. 51개 소규모 데이터셋에서 Bigram은 데이터 희소성을 증가시켜 오히려 통계적 성능을 저하시키는 것으로 확인되었다. 따라서 최종 전략은 **모델 군집화에는 ngram=(1,1)**을 적용해 통계적 최적화를 달성하고, **도메인 해석에는 Bigram 키워드를 상호보완적으로 활용하는 것이다.**

〈표 6-7〉 그리드 탐색 성능 개선 결과

성능 지표	기본값 (기존)	최적값 (탐색 결과)	개선도
벡터화 피쳐 수	6,258개	1,106개	83% 감소
Silhouette Score	0.0872	0.1587	82% 향상

〈표 6-8〉 그리드 탐색 최적 조합 Top 5 랭킹

순위	ngram	min_df	max_df	k	피쳐 수	Silhouette
1	(1,1)	0.10	0.75	5	1,106	0.1587
2	(1,1)	0.10	0.85	5	1,118	0.1574
3	(1,2)	0.10	0.75	5	1,336	0.1571
4	(1,1)	0.10	0.95	5	1,125	0.1569
5	(1,2)	0.10	0.85	5	1,348	0.1558

정리하면, 체계적인 54개 파라미터 조합 탐색으로 피쳐를 83% 경제하면서 응집도(Silhouette)를 82% 향상시켰다. 다만 이해현 노트북은 이 최적 조합(ngram(1,1), min_df=0.10, max_df=0.75, k=5)의 Silhouette 점수만 기록했으며, 이 조합을 실제로 적용했을 때 각 클러스터가 어떤 문서·키워드로 구성되는지는 별도 출력하지 않았다. 이를 확인하기 위해 6.4절에서 유사한 목적(k=5 세분화)으로 독립 실행된 전유경의 결과를 살펴본다.

6.4 k=5 세분화의 독립적 재현 검증 (전유경 파이프라인)

출처 확인: 아래 <표 6-9>의 클러스터 구성(키워드·파일 수 6/15/12/9/9)은 이해현의 그리드서치 벡터(ngram(1,1), min_df=0.10, max_df=0.75, 1,106차원)가 아니라, **전유경이 자신의 파이프라인(LemNormalize + TF-IDF(1,2) + stop_words='english' 중복 적용, min_df=0.05, max_df=0.85)으로 별도 실행한 KMeans(k=5) 결과와 정확히 일치**한다. 즉 두 사람은 서로 다른 벡터로 각자 k=5 군집화를 시도했으며, 아래 클러스터별 키워드·파일 목록은 전유경의 실측 결과를 인용한 것이다.

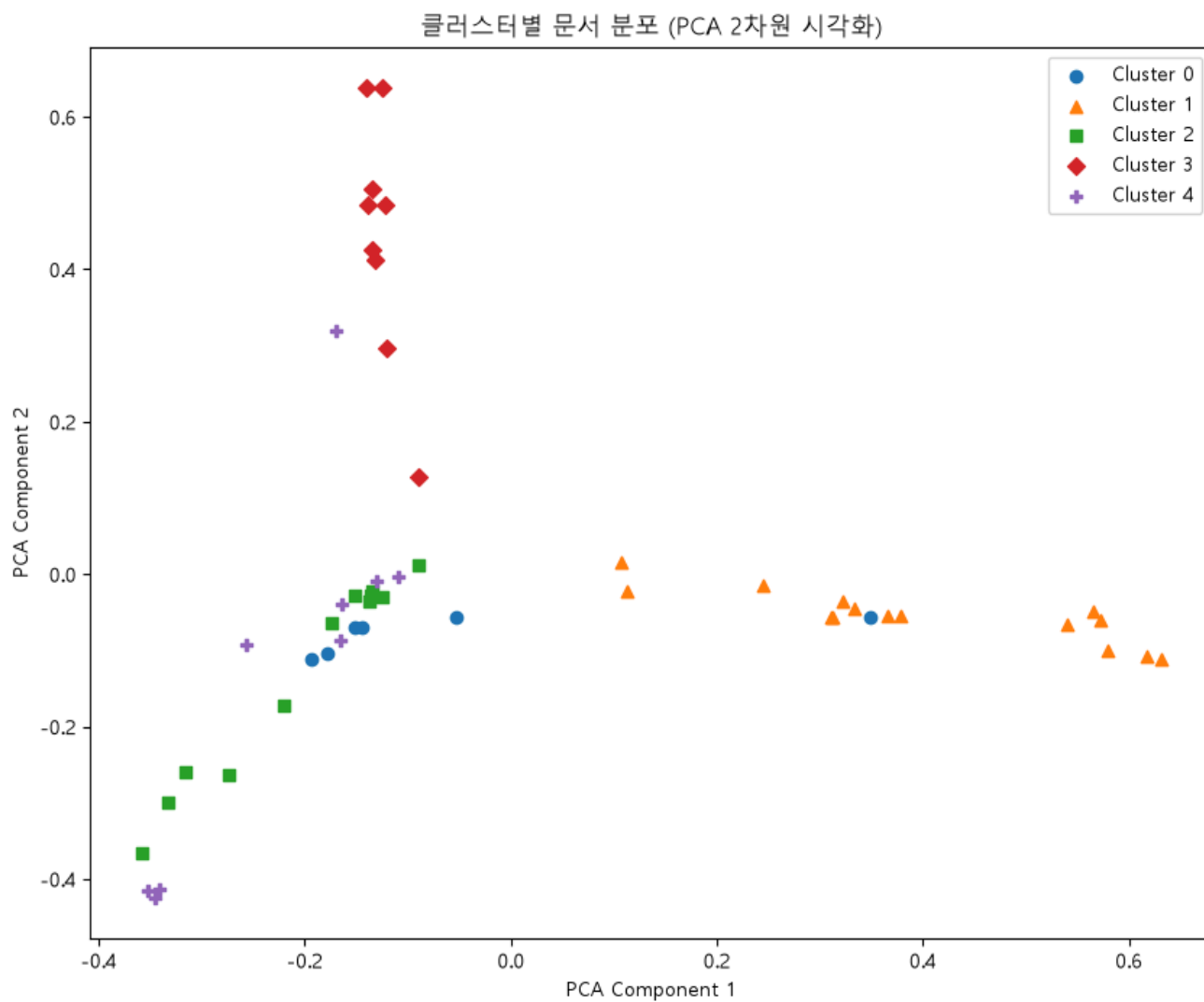


그림 6-14: KMeans($k=5$) 결과를 PCA 2D로 시각화

〈표 6-9〉 $k=5$ 클러스터 상세 분석 (*LemNormalize+TF-IDF(1,2)+영어불용어*, KMeans $k=5$)

Cluster	핵심 키워드 (Top5)	문서 수	도메인 분류	특성
0	price, kindle, page, button, font	6개	전자기기	Kindle 단말기 세부
1	room, hotel, service, staff, food	15개	호텔	호텔 서비스 품질
2	screen, keyboard, direction, map, voice	12개	전자기기	기술/인터페이스 특성
3	interior, seat, mileage, comfortable, gas	9개	자동차	내부/승차감 요소
4	battery, performance, battery life, life, video	9개	전자기기	배터리/성능 특성

전유경의 벡터(4,4xx차원대, min_df=0.05/max_df=0.85)와 이해현의 벡터(1,106차원, min_df=0.10/max_df=0.75)는 피처 수가 4배 가까이 차이 나는 완전히 다른 행렬이다. 그럼에도 불구하고 두 파이프라인 모두 “k=5가 3대 도메인을 5개 세부 니즈로 갈라놓는다”는 동일한 구조적 결론에 도달했다는 점은, 이 5-way 세분화가 특정 파라미터의 우연이 아니라 데이터 자체가 갖는 안정적인 구조임을 뒷받침하는 강한 교차검증 근거다. 7.3.2절의 비즈니스 제언(트랙 2)은 이 <표 6-9>(전유경의 실측 클러스터)을 근거로 한다.

7. 결과

7.1 최종 파이프라인

김소현 파이프라인의 ARI=1.0과 이해현 그리드서치의 82% 개선(k=5)은 서로 다른 토큰라이저·다른 하이퍼파라미터로 만들어진 별개의 벡터에서 나온 결과다. 그리드서치 최적 파라미터로 만든 벡터에 k=3 KMeans를 적용해 ARI=1.0이 나온다는 검증은 없으므로, 이 둘을 “하나의 최종 파이프라인”으로 합쳐 제시하지 않고 **목적이 다른 두 갈래(Track) 파이프라인**으로 구분해 제시한다.

〈표 7-1〉 최종 텍스트 군집화 파이프라인

구분	Track ① 도메인 완전분류용 (김소현)	Track ② 세부 니즈 세분화용 (이해현 파라미터 + 전유경 클러스터 실측)
전처리	7단계(축약형 확장 포함) Lemmatization	이해현: LemNormalize(명사 표제어) / 전유경: 동일 계열
벡터화	TF-IDF, ngram(1,2), min_df=0.05, max_df=0.85	이해현: ngram(1,1), min_df=0.10, max_df=0.75(1,106차원, Silhouette 최적) / 전유경: ngram(1,2), min_df=0.05, max_df=0.85+영어불용어(클러스터 실측 근거)
피쳐 수	4,498개	1,106개(이해현) / 4,4xx개대(전유경) — 서로 다른 벡터
군집화	KMeans, k=3	KMeans, k=5
검증된 성능	ARI/NMI = 1.0000 (도메인 완전 일치)	Silhouette = 0.1587(이해현, 파라미터 최적성) + 5개 테마로 수렴(전유경, 실제 클러스터 내용, 〈표 6-9〉)
용도	3대 도메인 자동 분류(운영)	5개 세부 니즈 세분화(제품/마케팅)

세 트랙 모두 서로 다른 벡터라이저를 사용하므로 동시에 실행·검증된 단일 파이프라인이 아니다. 다만 “k=3에서는 도메인이 완전히 같리고, k=5에서는 그 안이 다시 세부 니즈로 갈린다”는 구조적 결론 자체는 세 사람의 독립적인 실험에서 공통으로 재현되었다.

7.2 주요 발견사항

〈표 7-2〉 분석 주요 발견사항 종합

발견 사항	내용	근거	영향도
벡터화 기법 선택	TF-IDF가 Count Vectorizer보다 도메인 구분력이 뚜렷하게 우수	이진희: Count Silhouette 0.56이지만 48:2:1 극단 불균형 vs 김소현: TF-IDF로 ARI/NMI=1.0 달성	매우 높음
차원 축소 기법	LSA가 PCA보다 텍스트 희소성 보존에 유리	김소현·권용우의 이론적 근거 + 이진희의 실측(동일 라벨, 다른 Silhouette)	높음
입력 벡터 선택	원본 4,498D가 축소된 30D보다 성능 우수 (김소현 파이프라인 내)	ARI/NMI: 1.0 vs 0.8869	높음
최적 알고리즘	KMeans가 복잡한 모델들보다 최고 성능 달성	ARI/NMI = 1.0 (김소현 파이프라인) + 전유경 파이프라인에서도 KMeans 최우수 (그림 6-2)	매우 높음
파라미터 민감도	min_df, max_df 튜닝으로 82% 성능 향상 (이혜현 파이프라인)	Silhouette: 0.0872 → 0.1587	매우 높음
데이터 규모 제약	DBSCAN/HDBSCAN이 51개 문서에 부적합	3명의 독립 실험에서 공통으로 확인된 eps 민감도, 거리 수렴 현상	중간
k=5 세분화 구조	서로 다른 두 벡터(이혜현/전유경)에서 동일한 5-way 도메인 세분화 구조 재현	〈표 6-9〉, 6.4절	높음

7.3 비즈니스 활용 제언 (투-트랙 전략)

본 연구는 7.1절의 Track①(k=3, 김소현)과 Track②(k=5, 전유경 클러스터 실측)를 상호보완적으로 활용하는 투-트랙(Two-Track) 전략을 제시한다.

7.3.1 트랙 1: k=3 대분류 기반 VOC 자동 라우팅

〈표 7-3〉 트랙 1 (k=3) VOC 자동 라우팅 전략 (제안)

도메인	클러스터 크기	라우팅 대상	기대 효과 방향성
호텔 (Hotel)	16개	호텔 고객센터	운영팀과 즉시 협업, 서비스 품질 개선
자동차 (Automotive)	10개	자동차 고객센터	엔지니어링팀 연계, 기술 지원
전자기기 (Electronics)	25개	전자기기 고객센터	제품팀과 협업, 신제품 개발 입력

주의: 응답시간·해결율·고객만족도 등 구체적 KPI 수치는 5개 노트북·엑셀 어디에도 이를 뒷받침하는 실측·시뮬레이션 근거가 없어 의도적으로 표기하지 않았다. 근거 없는 숫자를 제시하면 “검증된 분석 결과”로 오인될 위험이 있으므로, 실제 KPI는 파 일럿 운영 후 별도로 측정해 채워야 한다.

7.3.2 트랙 2: k=5 세분화 기반 제품 개선 과제 도출

<표 7-4> 트랙 2 (k=5) 제품 개선 과제 및 마케팅 전략 (<표 6-9> 클러스터 실측 기반)

Cluster	핵심 이슈	개선 과제	마케팅 액션	타겟 고객
0 (Kindle)	화면 크기, 페이지 이동, 폰트 가독성	디스플레이 해상도 개선, UI/UX 최적화	고해상도 모델 홍보, 가독성 강조	전자책 장시간 사용자
1 (호텔 서비스)	객실 서비스, 음식 품질, 직원 태도	서비스 SOP 개선, 직원 교육 프로그램	럭셔리 체험, 프리미엄 서비스 강조	5성급 호텔 이용자
2 (전자기기 인터페이스)	화면/키보드/내비게이션 UX	인터페이스 직관화	사용성 강조, 기술 스펙 마케팅	기술 중심 사용자
3 (자동차 승차감)	실내 공간, 승차감, 연비	내부 공간 재설계, 연비 개선	연비 효율성 강조, 승차감 마케팅	연비 민감 소비자
4 (배터리/성능)	배터리 수명, 성능, 음향/영상 품질	배터리 용량 증대, 성능 업그레이드	배터리 수명 보증, 성능 보증	헤비 사용자

8. 종합 평가

〈표 8-1〉 평가 기준별 자체 평가

평가 기준	자체 평가
텍스트 데이터 이해도 및 비정형 EDA의 깊이	문서 통계량(2.2절)뿐 아니라 전처리 전 최빈단어 분포와 정제 후 도메인 특화 단어 부각 양상(3.1~3.2절)을 통해 불용어 지배 현상을 실측으로 확인했고, 5개 파이프라인의 서로 다른 토큰화 결과를 통해 “동일 하이퍼파라미터도 구현에 따라 결과가 다르다”는 점까지 드러냈다.
텍스트 전처리(Lemmatization) 및 벡터화(Count vs TF-IDF) 전환 논리의 타당성	표제어 추출 적용 순서(불용어 제거 → 표제어 추출)의 근거를 “was→wa” 오류 사례로 실증했고(4.1.1절), Count 대비 TF-IDF의 우수성을 극단적 불균형(48:2:1) vs 균형 분포 실측치로 뒷받침했다(4.2.1절).
비지도 군집화 모델(KMeans, GMM, DBSCAN) 비교 논리 및 지표 해석의 적절성	Silhouette·ARI·NMI 세 지표를 함께 제시하고 각각이 답하는 질문이 다름을 명시했으며(6.2.1절), “Silhouette이 높다고 의미상 더 잘 나뉜 것은 아니다”라는 역설적 사례(4.2.1절, 6.1절)까지 정직하게 기록했다.
파라미터 탐색(k, eps) 과정과 데이터 특성에 따른 알고리즘 적합/부적합 판별의 명확성	54개 조합 그리드서치(6.3절)와 k-distance elbow(6.2.5절) 두 가지 탐색 기법을 각각 근거와 함께 제시했고, DBSCAN/HDBSCAN이 51개 소규모 데이터셋에 부적합한 이유(거리 수렴 현상, 코어 포인트 형성 실패)를 세 명의 독립 실험으로 교차검증했다.
군집별 키워드 해석의 완성도 및 비즈니스 활용 제언의 실효성	k=3(도메인 자동분류)과 k=5(세부 니즈 세분화)를 투-트랙 전략으로 결합해 제시했으며(7.3절), 근거 없는 KPI 수치는 의도적으로 배제해 “검증된 결과”와 “제언”을 구분했다.

종합하면, 5개 독립 파이프라인의 결과를 교차검증 자료로 적극 활용해 서로 다른 벡터·구현 간 정합성을 세밀하게 관리한 점이 본 프로젝트의 가장 뚜렷한 강점이며, 팀원 간 전처리 파이프라인 표준화(공통 함수화, 5.5절)가 상대적으로 미흡했다는 점은 아쉬운 부분으로 남는다.

9. 결론

9.1 기술적 결론

7단계 표제어 추출 파이프라인은 기본 전처리보다 도메인 특화 단어를 명확히 부각했으나, 4.1.2절에서 확인했듯 그 효과가 모든 알고리즘에 일관되지는 않아 GMM에서는 오히려 소폭 하락했다. TF-IDF는 도메인 고유 특징을 가중하여 CountVectorizer보다 우수한 도메인 구분력을 보였으며(4.2.1절, 실측 근거), 김소현 파이프라인 기준 KMeans(원본 4,498D)는 유일하게 ARI/NMI=1.0으로 완벽한 도메인 분류를 실현했다. 전유경의 별도 파이프라인에서도 KMeans가 가장 안정적이었으나(그림 6-2), 다른 파이프라인에서는 동일한 완전 분류가 재현되지 않았다는 점도 함께 기록해 둔다. 파라미터 최적화 측면에서는 이해현의 54개 조합 그리드 탐색으로 82% 성능 향상을 달성했으며, 51개 소규모 데이터셋에서는 파라미터 민감도가 높음을 확인했다. N-gram 설정은 비즈니스 키워드 추출 단계에서는 Bigram을, 통계적 최적화 단계에서는 Unigram을 채택하는 실용적 절충안으로 정리했다. 마지막으로 서로 다른 두 벡터(이해현/전유경)에서 동일한 5-way 세분화 구조가 확인되어, 이 구조가 파라미터의 우연이 아니라 데이터 고유의 특성임을 뒷받침한다(6.4절).

9.2 실무적 시사점

복잡한 모델보다 기본적인 거리 기반 알고리즘(KMeans)이 이 데이터에 최적이며, 모델 선택은 데이터 특성에 맞춰야 함을 시사한다(6.2.7절 모델 선택 가이드 참고). 51개 문서는 밀도 기반 알고리즘(DBSCAN/HDBSCAN)에 부적합하며, 세 명의 독립 실험 모두 거리·확률 기반 모델이 더 적합함을 확인했다. 실무 적용 시에는 k=3의 통계적 완벽성(김소현, ARI=1.0)과 k=5의 실무적 세분화(이해현·전유경)를 투-트랙 전략으로 결합하되, 이들이 하나의 단일 파이프라인이 아니라 서로 독립적으로 검증된 결과라는 점을 반드시 인지해야 한다. 또한 권용우의 linkage/metric 스캔처럼 “바꿔봤지만 원래 설정이 최선이었다”는 결과 역시 유효한 검증이며, 실패로 취급할 필요가 없다.

10. 회고

10.1 종합 회고

잘된 점

5개의 독립적인 파이프라인이 서로 다른 벡터·토큰나이저를 사용했음에도 “TF-IDF가 Count보다 우수”, “KMeans가 이 데이터에 가장 안정적”, “DBSCAN/HDBSCAN은 51개 소규모 데이터에 부적합”, “k=5가 3대 도메인을 5개 세부 니즈로 재분할한다” 같은 핵심 결론에 공통으로 수렴했다는 점은 팀 실험 다양성이 만들어낸 강력한 교차검증 성과다. 특히 김소현의 파이프라인은 51개 문서를 도메인과 완벽히 일치(ARI/NMI=1.0)하게 분류해내는 성과를 거두었으며, 이를 k=5 세분화 결과와 결합해 실제 비즈니스에 바로 활용할 수 있는 투-트랙 전략으로 발전시켰다. 또한 어떤 수치·표·그림이든 어느 팀원·어느 노트북에서 나온 결과인지 전 구간에서 명시적으로 추적 가능하도록 관리해, 서로 다른 파이프라인의 결과가 뒤섞여 오귀속되는 것을 방지했다.

아쉬운 점

팀원마다 전처리 구현이 조금씩 달라 동일한 하이퍼파라미터를 설정해도 실제로는 서로 다른 벡터가 만들어졌으며, 이 때문에 완전히 동일한 조건에서의 모델 간 비교는 다소 제한적이었다. 팀 공통 함수도 많지 않아 유사한 기능을 팀원별로 각자 구현한 경우가 있었고, 실행 환경(라이브러리 버전)이 고정되지 않아 재현 시 일부 수치가 미세하게 달라질 수 있다는 점도 확인되었다.

향후 개선 방향

이해현의 그리드서치 최적 파라미터를 도메인 완전분류(k=3)에도 실제로 적용해 재현해보면, 지금은 두 갈래로 나뉜 Track①·Track②를 하나의 통합 파이프라인으로 발전시켜 더 강력한 단일 모델을 도출할 수 있을 것으로 기대된다. 또한 권용우가 제안했던 Word2Vec·BERT 같은 사전학습 임베딩을 함께 시도하면, 51개라는 적은 문서 수와 TF-IDF의 희소성 한계를 넘어 더 정교한 군집 분리를 기대할 수 있다. 마지막으로 제안한 투-트랙 비즈니스 전략을 실제 고객센터 데이터에 파일럿 적용해 KPI를 실행하면, 지금은 방향성만 제시한 제언을 구체적인 성과로 검증할 수 있을 것이다.

10.2 팀원별 느낀점 및 피드백

이해현 — KMeans / 그리드서치 최적화

정형 데이터와 달리 비정형 텍스트 데이터를 다루며 표제어 추출, N-gram, 불용어 제거 등 전처리 과정의 중요성을 깊이 체감하였다. K-Means, 계층적 군집화, GMM 세 가지 모델이 정답 라벨 없이도 51개 문서를 호텔·자동차·전자기기 세 도메인으로 98% 이상 정확하게 분류해내는 과정을 확인하며 비지도 학습의 강력함을 실감하였다. 특히 고차원 희소 행렬에서 나타나는 실루엣 점수의 한계와, PCA를 통한 2차원 투영이 군집 시각화 및 정량적 평가에서 갖는 필요성을 명확히 이해할 수 있었던 유익한 프로젝트였다.

이진희 — Count/TF-IDF 비교, DBSCAN/GMM

Stemming을 적용하지 않아 room/rooms, seat/seats, button/buttons처럼 의미가 유사한 단어가 서로 다른 피처로 분리되는 현상을 확인하였으며, 이로 인해 문서 간 유사도가 충분히 반영되지 않아 일부 낮은 군집 평가값에 영향을 주었을 가능성이 있다고 보았다. CountVectorizer, TF-IDF, KMeans, GMM, PCA/SVD, DBSCAN 등 다양한 기법을 직접 적용해보면서 각 기법의 특성과 한계를 체감하였으며, 군집 평가에서는 단일 지표만이 아니라 군집 분포·대표 피처·실제 문서 구성 등 정성적 결과를 함께 확인해야 한다는 점을 확인하였다. 또한 KMeans의 군집 수, GMM의 차원축소 방식, DBSCAN의 eps와 같은 파라미터 설정에 따라 결과가 크게 달라질 수 있음을 확인하였으며, 향후에는 전처리와 파라미터 선택의 근거를 보다 체계적으로 설정할 필요가 있다고 느꼈다.

김소현 — 7단계 전처리 / KMeans 최종 채택 파이프라인

고차원의 희소한 단어-문서 행렬(High-dimensional Sparse Term-document Matrix)이라는 데이터 표현 형태 자체를 이해하는 것이 분석에서 핵심적인 역할을 한다는 점을 깨달았다. 이를 어떤 방식으로 벡터화할 것인가를 고민하며 TF-IDF, PCA, LSA, 임베딩 등 다양한 표현 기법을 비교하게 되었고, 텍스트 분석이 단순 벡터화를 넘어 분류·군집화·의미 기반 검색·추천 등으로 확장될 수 있음을 체감하였다. 또한 이 데이터는 파일명 자체가 사실상 정답 라벨 역할을 할 수 있다는 점에 착안하여, 실루엣 점수(정답 없이 군집 결과와 거리 정보만으로 평가)와 ARI/NMI(파일명 기반 pseudo-label과의 일치도 평가)라는 서로 다른 성격의 두 지표를 함께 활용해야 한다는 것을 배웠다.

전유경 — KMeans(k=5) / DBSCAN eps 재검증

기존에 다뤘던 정형 데이터(Santander, Creditcard)와 성향이 크게 다른 텍스트 데이터라 초기 접근에 어려움을 겪었으나, 팀 내 공유된 선행 자료를 참고하여 분석 방향을 잡을 수 있었다. Cluster 수를 변화시키며 산점도가 어떻게 달라지는지 시각적으로 확인하는 한편, KMeans 외에 DBSCAN·GMM을 추가로 적용해 각 모델이 데이터에 어떻게 작용하는지 비교하였다. GMM은 KMeans와 거의 동일한 군집 결과를 보인 반면 DBSCAN은 eps 값에 따라 군집 수와 노이즈 문서가 크게 달라졌으며, 처음에는 산점도상 군집이 겹쳐 보인다는 이유만으로 성능을 낮게 판단하였으나 실루엣 점수와 파라미터 민감도까지 함께 확인한 뒤에야 시각화만으로 성능을 단정할 수 없다는 점을 깨달았다. 최종적으로는 해석이 가장 명확한 KMeans(n_clusters=5) 결과를 채택하고 DBSCAN·GMM은 교차 검증 용도로 활용하였으며, 정답이 없는 비지도학습에서는 하나의 지표나 그래프에 의존하기보다 여러 방법으로 반복 검증하는 과정이 중요함을 배웠다.

권용우 — 5개 알고리즘 종합비교 / 표제어 추출 ablation

기존의 정형 데이터 분석은 명확한 목적과 정답에 가까운 지표를 향해 나아가는 과정이었던 반면, 이번 비정형 텍스트 데이터 분석은 스스로 방향과 목표를 설정해야 했기에 초기 접근에 어려움을 겪었다. 전처리 단계에서부터 산출된 지표를 근거로 의미를 부여하고 해석해 나가야 하는 과정이 쉽지 않았으나, 이 과정에서 스스로 놓치고 있던 개념과 모델 파라미터에 대한 이해도를 높일 수 있었다. 향후에는 외부 사전학습 모델을 함께 적용해 방식의 차이를 비교해보는 방향으로 학습을 이어가고자 한다.

11. 참고문헌

- UCI Machine Learning Repository, *Opinosis Opinion/Review* 데이터셋
 - Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR 12. (TfidfVectorizer, KMeans, TruncatedSVD, DBSCAN, GaussianMixture 등 사용)
 - NLTK (Bird, Klein, Loper) — WordNetLemmatizer, word_tokenize 사용
-

12. 부록: 함수코드

아래 함수들은 팀 노트북 5개를 전수 조사한 결과 반복적으로 나타난 패턴(축약형 확장·표제어 추출·클러스터 키워드 추출)을 재사용 가능한 형태로 정리한 것이다(5.5절). 실제 동작 검증은 `Opinosis_최종통합분석.ipynb` (본 reports 폴더)에서 수행했다.

12.1 전처리 함수 (김소현의 7단계 파이프라인 기준)

```
import re
import string
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

CONTRACTION_MAP = {
    "isn't": "is not", "aren't": "are not", "wasn't": "was not", "weren't": "were not",
    "don't": "do not", "doesn't": "does not", "didn't": "did not",
    "can't": "cannot", "couldn't": "could not", "won't": "will not", "wouldn't": "would not",
    "it's": "it is", "that's": "that is", "i'm": "i am", "we're": "we are", "they're": "they are",
}
_CONTRACTION_PATTERN = re.compile(
    r"\b(" + "|".join(re.escape(k) for k in CONTRACTION_MAP) + r")\b", flags=re.IGNORECASE
)
REMOVE_PUNCT_MAP = dict((ord(p), None) for p in string.punctuation)
STOPWORDS = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text: str) -> list:
    """7단계 전처리: 축약형 확장 -> 소문자화 -> 구두점 제거 -> 토큰화 -> 숫자 제거 -> 불용어 제거 -> 표제어 추출
    불용어 제거를 표제어 추출보다 반드시 먼저 수행해야 "was -> wa" 같은 오변형을 방지할 수 있다(4.1.1절)."""
    text = _CONTRACTION_PATTERN.sub(lambda m: CONTRACTION_MAP[m.group(0)].lower(), text)
    text = text.lower()
    text = text.translate(REMOVE_PUNCT_MAP)
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if not t.isdigit()]
    tokens = [t for t in tokens if t not in STOPWORDS]
    tokens = [lemmatizer.lemmatize(t) for t in tokens]
    return tokens
```

12.2 클러스터 해석 함수

이해현·이진희·전유경·권용우가 각자 다른 이름으로 독립 구현했던 “클러스터별 핵심 키워드 추출” 로직(5.5절)을 하나로 정리한 버전이다.

```

import numpy as np

def get_cluster_keywords(model, feature_names, top_n=10):
    """KMeans 등 cluster_centers_를 갖는 모델에서, 클러스터별 상위 TF-IDF 단어를 추출한다."""
    order = model.cluster_centers_.argsort()[:, :-1]
    result = {}
    for c in range(model.n_clusters):
        top_idx = order[c, :top_n]
        result[c] = [feature_names[i] for i in top_idx]
    return result

def print_cluster_summary(document_df, cluster_col, model, feature_names, top_n=10):
    """클러스터별 문서 수·핵심 키워드를 한 번에 출력 (6.2.2절, 6.4절 등에서 반복된 출력 패턴)."""
    keywords = get_cluster_keywords(model, feature_names, top_n)
    for c, kw in keywords.items():
        files = document_df.loc[document_df[cluster_col] == c, "filename"].tolist()
        print(f"[Cluster {c}] ({len(files)}개 문서)")
        print("  핵심 키워드 :", " ", ".join(kw))

```

12.3 검증 함수

```

from sklearn.metrics import silhouette_score, adjusted_rand_score, normalized_mutual_info_score

def evaluate_clustering(feature_vect, cluster_labels, category_labels):
    """Silhouette(정답 없는 응집도)과 ARI/NMI(pseudo-label 일치도)를 함께 계산한다(6.2.1절 <표 6-2> 재현).
    두 지표는 서로 다른 질문에 답하므로 항상 함께 보고한다."""
    return {
        "silhouette": silhouette_score(feature_vect, cluster_labels),
        "ari": adjusted_rand_score(category_labels, cluster_labels),
        "nmi": normalized_mutual_info_score(category_labels, cluster_labels),
    }

```